AN AI ECOSYSTEM FOR FAKE JOB SCAM DETECTION AND FRAUD-AWARE JOB RECOMMENDATION

Kajanthan U

(IT22224002)

BSc (Hons) degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

AN AI ECOSYSTEM FOR FAKE JOB SCAM DETECTION AND FRAUD-AWARE JOB RECOMMENDATION

INDIVIDUAL THESIS

Kajanthan U

(IT22224002)

Individual component: Scam Communication Detection (BSTD)

Group ID: R26-SE-002

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Kajanthan U

Student ID: IT22224002

Signature: U. kajanthan Date: 11/102026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Karthiga Rajendran		
Co-supervisor: Ms. Thilini Jayalath		

ABSTRACT

Online recruitment scams rarely end at the advertisement. After the first contact, fraudsters move the job seeker to private channels such as WhatsApp, Telegram, SMS or email, where they use psychological pressure to obtain money and personal documents. Existing detection systems examine the advertisement only, return a binary scam or legitimate label, work on a single platform, and do not tell the job seeker which manipulation tactic is being used. This research aimed to design and evaluate the Behaviour-Inspired Semantic Tactic Detection (BSTD) component, which analyses recruiter-candidate messages and turns the four-tactic taxonomy of urgency pressure, social proof, fear of missing out (FOMO) and sunk-cost influence into a working classifier. Phase 1 fine-tunes DistilBERT for binary scam classification on 7,433 messages drawn from EMSCAD, the UCI SMS Spam Collection, synthetic messages and real recruiter messages, using a stratified 80/10/10 split. Phase 2 initialises a multi-label tactic classifier from the Phase 1 encoder (intermediate-task transfer) and fine-tunes it on 3,840 messages using a group-aware 70/10/20 split, with per-tactic thresholds chosen on validation data. Messages arrive through a built-in live chat, pasted text, or screenshots read by EasyOCR with a Tesseract fallback, SHAP and LIME attributions are mapped to plain-language warnings, and a combination rule gives a three-tier verdict of scam, suspicious or legitimate. On held-out data the binary classifier reached 93.82% accuracy (scam-class F1 0.9065, n = 744), and the tactic classifier reached a macro F1 of 0.8345, a micro F1 of 0.8722 and an exact-match accuracy of 85.36% (n = 758). Urgency pressure and social proof exceeded an F1 of 0.90, while FOMO was the weakest tactic (F1 0.6395). The contribution is the first automated, explainable tactic-level detector for recruitment-scam conversations, which also outputs a binary communication-scam indicator that the fraud-aware job recommender is designed to accept.

Keywords: scam communication detection, recruitment fraud, manipulation tactics, DistilBERT, multi-label classification, explainable AI

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Ms. Karthiga Rajendran**, for her continuous support, insightful guidance, and encouragement throughout this research. Her expertise and constructive feedback, especially on the tactic taxonomy, the dataset design and the evaluation of the tactic classifier, were instrumental in shaping the direction and quality of this work.

I am also deeply grateful to my co-supervisor, **Ms. Thilini Jayalath**, for her valuable advice and constant motivation, which helped me overcome challenges and achieve the research objectives. I sincerely thank **Mr. Saman Gunawardana**, the external validator, for reviewing this research and offering helpful industry-oriented suggestions.

I thank my group members, Nipuni Tharushika Bandara, Sirithar Santhosh, and Chethya Methmi, for their cooperation and for helping integrate my component with the other components of the system. I declare that the design, implementation, and evaluation of the Scam Communication Detection (BSTD) component and the preparation of this dissertation are my own work. I used an AI assistant (Claude, by Anthropic) for guidance on evaluation design, for checking analysis scripts, and for language editing, and I verified all results myself.

Heartfelt appreciation is extended to the **Department of Software Engineering, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT)**, for providing the necessary facilities and an inspiring research environment. This research received no external funding. Finally, I wish to thank my family and friends for their unwavering support, encouragement, and understanding throughout this journey.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 1
1.3 Problem Statement 3
1.4 Research Gap 3
1.5 Aim and Objectives 4
1.6 Research Questions or Hypotheses 5
1.7 Scope, Limitations and Assumptions 6
1.8 Individual Contribution 7
1.9 Thesis Structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 10
2.3 Related Research 13
2.4 Existing Systems, Products and Techniques 15
2.5 Gap Synthesis and Conceptual Framework 16
3 RESEARCH METHODOLOGY 19
3.1 Research Design 19
3.2 Data, Participants or Study Materials 20
3.3 Experimental or Study Procedure 22
3.4 Measures and Evaluation Metrics 24
3.5 Data Analysis 25

3.6 Ethics, Privacy, Security and Responsible AI 26
3.7 Reproducibility and Research Artefacts 27
4 PRODUCT AND SOLUTION DESIGN 29
4.1 Requirements 29
4.2 Component Architecture within the Complete Solution 30
4.3 Design Alternatives and Decisions 33
4.4 Detailed Design 35
4.5 Standards and Professional Practice 42
5 IMPLEMENTATION, INTEGRATION AND TESTING 43
5.1 Development Environment and Technology Stack 43
5.2 Component Implementation 44
5.3 Integration with the Other Components 46
5.4 Verification and Software Testing 47
5.5 Deployment and Operation 52
6 RESULTS AND VALIDATION 53
6.1 Research Results 53
6.2 Product and Technical Results 57
6.3 User or Stakeholder Validation 58
6.4 Integration Results 59
6.5 Summary of Findings 62
7 DISCUSSION 64
7.1 Interpretation of Findings 64
7.2 Comparison with Literature Baselines and Existing Solutions 65
7.3 Research Contribution 66

7.4 Product Contribution 67
7.5 Limitations and Threats to Validity 67
7.6 Implications 69
7 CONCLUSION AND FUTURE DIRECTIONS 70
8.1 Conclusion 70
8.2 Achievement of Objectives 71
8.3 Individual Research and Product Contributions 71
8.4 Future Directions 72
REFERENCES 73
GLOSSARY 75
APPENDICES 76
Appendix A Extended Results and Technical Specification 76
Appendix B Academic Integrity and AI Use Disclosure 77

LIST OF FIGURES

Figure 2.1: Conceptual framework of the BSTD component 18
Figure 3.1: Research procedure from data to final evaluation 23
Figure 4.1: Position of the BSTD component in the complete solution 32
Figure 4.2: Endpoints of the BSTD service on the generated API documentation page 35
Figure 4.3: A stored check in the scans collection (personal data hidden) 40
Figure 4.4: Folder structure of the scam-detection service 41
Figure 5.1: Detect section of the mobile application and the list of recent checks 46
Figure 5.2: Output of the automated test run (18 passed, 2 skipped) 50
Figure 6.1: Phase 1 precision, recall and F1-score by class on the test split 54
Figure 6.2: Phase 2 precision, recall and F1-score by tactic on the test split 55
Figure 6.3: Warning shown to the job seeker in the built-in chat and the matching notification 60
Figure 6.4: Results of a legitimate and a scam message in the Detect section 61
Figure 6.5: Checking a screenshot: the uploaded image and the result 62

LIST OF TABLES

Table 1.1: Research questions 5
Table 2.1: Scope of the review 10
Table 2.2: Comparison of the closest related studies with this research 14
Table 2.3: Existing products for detecting scam messages 15
Table 3.1: Research questions and the methods used to answer them 19
Table 3.2: Composition of the datasets 21
Table 3.3: Training settings 24
Table 3.4: Per-tactic decision thresholds chosen on the validation split 24
Table 3.5: Evaluation measures 25
Table 3.6: Objectives, measures and evidence 26
Table 3.7: Research artefacts 28
Table 4.1: Functional requirements 29
Table 4.2: Non-functional requirements 30
Table 4.3: Interfaces of the component 33
Table 4.4: Design decisions 33
Table 4.5: Tools and technologies 34
Table 4.6: Endpoints of the BSTD service 35
Table 4.7: Rules of the combination function 38
Table 4.8: Warning texts shown to the user 39
Table 4.9: Fields of a document in the scans collection 40
Table 4.10: Main files and folders of the scam-detection service 41
Table 5.1: Development environment 43
Table 5.2: Main settings of the services (secrets removed) 44
Table 5.3: Time and size limits 45
Table 5.4: Automated tests of the service 48
Table 5.5: Scenario tests of the service 51
Table 6.1: Phase 1 results on the test split 53
Table 6.2: Phase 2 results on the test split 54

Table 6.3: Baseline model results on the external test set..... 56
Table 6.4: Outcome of the hypotheses..... 56
Table 6.5: Technical results of the service..... 57
Table 6.6: Feedback from the external validator and the response of the team..... 59
Table 6.7: Summary of findings 62
Table 0.1: Comparison with the closest studies and products 65
Table 7.1: Achievement of the objectives of Section 1.5.2 71

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BSTD	Behaviour-Inspired Semantic Tactic Detection
CCID	Computer Crimes Investigation Division
DistilBERT	Distilled version of BERT
EMSCAD	Employment Scam Aegean Dataset
F1	Harmonic mean of precision and recall
FOMO	Fear of Missing Out
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
ML	Machine Learning
MMS	Multimedia Messaging Service
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORF	Online Recruitment Fraud
RCS	Rich Communication Services

Abbreviation	Meaning
REST	Representational State Transfer
SHAP	SHapley Additive exPlanations
SMS	Short Message Service
TF-IDF	Term Frequency-Inverse Document Frequency
TOPSIS	Technique for Order of Preference by Similarity to Ideal Solution
UCI	University of California, Irvine

1 INTRODUCTION

1.1 Overall Project Context

Online recruitment fraud reaches a job seeker at several points: the job advertisement, the employer behind it, the conversation with the recruiter, and the choice of which jobs to apply for. Most existing tools protect only one of these points, usually by classifying the text of an advertisement. The group project R26-SE-002 therefore proposes an AI ecosystem of four connected components, each producing an independent risk signal.

- Fake job post detection: an XLM-RoBERTa model classifies job advertisements and gives a three-tier suspicion score explained with SHAP and LIME.
- Employer legitimacy verification: website analysis, Sri Lankan registry checks and reputation aggregation are combined with a Random Forest score into a weighted legitimacy score and risk band.
- Scam communication detection (BSTD): the component reported in this dissertation, which detects scam messages and manipulation tactics in recruiter-candidate conversations.
- Fraud-aware job recommendation: TF-IDF skill matching is combined with the risk signals of the other components, and jobs are ranked with the Entropy Weight Method and TOPSIS.

The outputs of the three detection components are designed to feed the recommendation component. Each group member designed, implemented and evaluated one component.

1.2 Individual Component Context

A recruitment scam usually runs in two phases. In the first phase a deceptive advertisement attracts the victim; in the second, the "recruiter" moves the conversation

to WhatsApp, Telegram, SMS or email and asks for registration fees, processing charges, document copies or bank details. The advertisement and employer checks of the other components cannot see this second phase, which is where the money is actually lost. The scam communication component, called Behaviour-Inspired Semantic Tactic Detection (BSTD), analyses these recruiter-candidate messages. It accepts messages in three ways: through the built-in recruiter-candidate chat of the mobile application, as text pasted from any external platform, or as a screenshot whose text is read by EasyOCR, with Tesseract as a fallback when EasyOCR returns too little text. No access to the API of any messaging platform is needed.

Every message passes through two stages. Phase 1 fine-tunes DistilBERT for binary scam classification on 7,433 messages drawn from EMSCAD, the UCI SMS Spam Collection, synthetic messages and real recruiter messages, using a stratified 80/10/10 split. Phase 2 initialises a multi-label tactic classifier from the Phase 1 encoder (intermediate-task transfer) and fine-tunes it on 3,840 messages using a group-aware 70/10/20 split, to identify which of the four manipulation tactics described by Anagha et al. [1] are present: urgency pressure, social proof, fear of missing out (FOMO) and sunk-cost influence. Each tactic has its own decision threshold, chosen on validation data, and a tactic is flagged when its probability exceeds its threshold.

SHAP and LIME word attributions are mapped to plain-language warnings, for example that the recruiter is creating artificial time pressure, so that a non-technical job seeker understands why the message is risky. A combination rule then gives a three-tier verdict: SCAM when any tactic is detected or the Phase 1 scam probability is at least 0.75, SUSPICIOUS when the probability is between 0.40 and 0.75, and LEGITIMATE otherwise. If FOMO is the only tactic detected and the Phase 1 scam probability is below 0.40, the message is classed as LEGITIMATE, so that ordinary HR updates that mention other candidates are not flagged.

The BSTD service exposes REST endpoints through FastAPI (/classify, /classify-image, /ocr-only and /history). The chat-management service (Node.js with Socket.IO) calls it for each text message sent by a recruiter to a job seeker; a message classed as

SCAM is shown to the job seeker with a warning, and a SUSPICIOUS message with a softer caution. Only the job seeker sees these, and messages are never blocked. The component also outputs a binary communication-scam indicator that the recommendation component is designed to accept.

1.3 Problem Statement

Job seekers have no easy and reliable way to recognize manipulation while they are talking to a recruiter. Once a conversation leaves the job platform, the protections of the platform no longer apply, and scammers use deadlines, fake authority or endorsements to appear legitimate, fear of losing the opportunity, and requests for "one more" payment after an earlier one to push victims into acting before they think. Pitumpe and Rahmati [2] found that scammers reuse structured message templates across SMS, WhatsApp and Telegram, which suggests that these patterns can be learned, yet automated tools treat such messages as a binary spam problem and give no explanation. Automated approaches therefore leave open the following problem: there is a need for a cross-platform method that detects scam messages in recruiter-candidate conversations, identifies which manipulation tactic is being used, and explains the result in language that a non-technical job seeker can act on, with its accuracy measured on held-out data.

1.4 Research Gap

Ghourabi [3] showed that a BERT-based model detects fraudulent SMS messages more reliably than keyword filters. Shinde et al. [4] showed that OCR can extract scam text from screenshots, so that classification does not depend on platform access. Anagha et al. [1] identified four recurring manipulation tactics in recruitment scam conversations. Taken together, these studies show that conversation-level scam detection is feasible, but they leave four gaps:

1. Advertisement-only focus: fake job detection research, mostly on the EMSCAD dataset, analyses the advertisement and gives no protection once communication moves to private channels.
2. Binary output: message classifiers label a message as scam or legitimate but do not say which manipulation tactic is being used; the tactic taxonomy of Anagha et al. [1] has remained theoretical and has not been implemented as an automated classifier.
3. Single-platform input: existing systems work on one channel, such as SMS, and cannot accept messages pasted or captured from other platforms.
4. Explainability for the end user: where explanations exist, they target security analysts rather than the job seeker who has to decide whether to reply or pay.

1.5 Aim and Objectives

1.5.1 Aim

The aim is to design, implement and evaluate an explainable, cross-platform component that detects scam messages in recruiter-candidate conversations and identifies the manipulation tactics they use, with its performance measured on held-out data.

1.5.2 Objectives

The objectives are:

O1. Review related work on scam message detection, OCR-based extraction and manipulation tactics in recruitment fraud and identify the gaps (Section 1.4 and Chapter 2).

O2. Build a binary scam dataset and a four-tactic multi-label dataset of recruitment messages, with a stratified split for Phase 1 and a group-aware split for Phase 2.

O3. Fine-tune a DistilBERT model for binary scam classification and measure it on held-out data.

O4. Initialise a multi-label tactic classifier from the Phase 1 encoder (intermediate-task transfer), tune a threshold for each tactic on validation data, and measure per-tactic and overall performance on held-out data.

O5. Support three input modes (live chat, pasted text and screenshot OCR) and convert SHAP and LIME attributions into plain-language warnings.

O6. Expose the component as a REST service, connect it to the real-time chat of the mobile application, and provide a binary communication-scam indicator for the recommendation component.

1.6 Research Questions or Hypotheses

The research is guided by the four questions listed in Table 1.1.

1.6.1 Research Questions

Table 1.1: Research questions

No.	Research question
RQ1	How can a recruitment-message dataset with binary scam labels and four tactic labels be built so that related messages do not leak between training and test data?
RQ2	How well can a fine-tuned DistilBERT model separate scam from legitimate recruiter messages on data it has not seen?
RQ3	How accurately can urgency pressure, social proof, FOMO and sunk-cost influence be detected by a multi-label head that reuses the Phase 1 encoder, and which tactics are hardest to detect?
RQ4	How can messages from live chat, other platforms and screenshots be handled by one pipeline and explained to job seekers as plain-language warnings?

1.6.2 Hypotheses

To make the evaluation testable, the following working hypotheses were set before the experiments were run.

- H1. The fine-tuned DistilBERT model reaches an F1-score of at least 0.85 for the scam class on the held-out binary test set.

- H2. The multi-label tactic classifier with per-tactic thresholds reaches a macro F1 of at least 0.80 on the held-out tactic test set.
- H3. Tactics with explicit lexical markers and more training examples (urgency pressure and social proof) are detected more reliably than tactics expressed in more varied language (FOMO and sunk-cost influence).

The results chapter reports whether each hypothesis was supported. The proposal set a higher target F1 of 0.90; the difference between that target and the measured results is discussed in Chapter 7.

1.7 Scope, Limitations and Assumptions

1.7.1 Scope

The component analyses recruiter-candidate messages written in English, received through the built-in chat, pasted from external platforms, or captured as screenshots. It returns a scam decision, the detected tactics and a plain-language warning; it is a risk estimate and not a legal ruling. The evaluation covers the binary and tactic classifiers; the other components are outside the scope.

1.7.2 Limitations

Real recruitment-scam conversations are hard to collect, so only 718 of the binary messages are manually collected real messages from Reddit, LinkedIn and Facebook; the rest come from EMSCAD postings, the UCI SMS Spam Collection and reviewed synthetic messages, which differ in style, and results describe controlled offline performance. Each message is classified on its own, so escalation across a whole conversation is not modelled. FOMO has the lowest recall (0.54). The Phase 1 split was not deduplicated or grouped, the Phase 2 test set contains back-translated copies of some messages, and Phase 2 test messages were not excluded from Phase 1 training, so the reported results may be optimistic; a retrained, leakage-free evaluation is planned. Messages that mix legitimate phrasing with subtle red flags can be

misclassified, which shows a reliance on surface lexical cues. The reported metrics evaluate each model separately, not the combined three-tier verdict. OCR accuracy was not evaluated separately, and Sinhala, Tamil and code-mixed messages are not covered.

1.7.3 Assumptions

The four tactics of Anagha et al. [1] cover the main manipulation patterns in recruitment scams; the labels in the dataset are correct; and text extracted from screenshots is close enough to typed text to be classified by the same model.

1.8 Individual Contribution

- Collected 718 real recruiter messages and prepared the binary scam dataset (7,433 messages, stratified 80/10/10 split) and the four-tactic multi-label dataset (3,840 messages, group-aware 70/10/20 split).
- Designed and trained the two-phase DistilBERT model: binary scam classification in Phase 1 and a multi-label tactic classifier initialised from the Phase 1 encoder (intermediate-task transfer) in Phase 2, with per-tactic thresholds tuned on validation data.
- Built the screenshot pipeline with EasyOCR as the primary engine and Tesseract as a fallback when EasyOCR returns too little text, and designed the three-tier SCAM, SUSPICIOUS and LEGITIMATE combination rule.
- Mapped SHAP and LIME attributions to plain-language warning templates for job seekers.
- Exposed the component as REST endpoints through FastAPI. The chat-management service (Node.js with Socket.IO) calls it for each text message from a recruiter to a job seeker and warns the job seeker when the message is classed as SCAM or SUSPICIOUS.

- Evaluated both phases on held-out test sets (744 binary and 758 tactic messages) using accuracy, per-class and per-tactic precision, recall and F1, macro and micro F1, Hamming loss and exact-match accuracy.

Link to the group solution. The component receives messages from the built-in chat and from the Detect section of the mobile application, and returns the three-tier verdict, the detected tactics and a warning. It also outputs a binary communication-scam indicator that the recommendation component is designed to accept as one of its risk signals. The main findings are reported in Chapter 6.

1.9 Thesis Structure

Chapter 2 reviews the literature and ends with a conceptual framework. Chapter 3 describes the research methodology. Chapter 4 presents the product and solution design, and Chapter 5 the implementation, integration and testing. Chapter 6 reports the results and validation, and Chapter 7 discusses them. Chapter 8 gives the conclusion and future directions. The dissertation ends with the references, a glossary and the appendices.

2 LITERATURE REVIEW AND RELATED WORK

This chapter reviews the research and the existing practice that bear on scam communication detection in recruitment. It explains how the review was carried out, sets out the domain and theoretical foundations that the BSTD component relies on, examines related research and existing products, and ends by synthesizing the gaps into a conceptual framework that guides the methodology in Chapter 3.

2.1 Review Method and Scope

The review looked for work that helps answer the research questions in Section 1.6: how recruitment scams are carried out in private conversations, which manipulation tactics they use, how scam messages can be detected automatically, and how such detectors should be built, explained and evaluated. Three strands were reviewed: (a) online recruitment fraud and scam message detection, (b) behavioural manipulation tactics, and (c) the methods used in this component, namely transformer language models, intermediate-task transfer, multi-label classification, optical character recognition, explainability and leakage-aware evaluation.

Searches were conducted in Google Scholar, IEEE Xplore and arXiv using the terms "online recruitment fraud", "job scam", "fake job detection", "smishing detection", "scam conversation", "manipulation tactics" and "multi-label text classification", mainly for the period 2017 to 2026. Studies were included when they addressed recruitment fraud, scam or phishing message detection, scam behaviour, or a method used directly in this dissertation. Studies that only classified job advertisements were included as background, not as direct comparators, because they do not analyse conversations. Existing products were reviewed from their official announcements and reputable technology news. Every source cited in this chapter was checked against its published record. No published study was found that detects manipulation tactics in recruiter-candidate conversations in the Sri Lankan context; this is stated as a finding

of this review and not as proof that none exists. Table 2.1 summarizes the scope of the review.

Table 2.1: Scope of the review

Strand	Question asked	Used in
Recruitment fraud and scam message detection	How do recruitment scams reach the victim, and what has been automated?	Sections 2.2.1, 2.3.1, 2.3.2 and 2.4
Behavioural manipulation tactics	Which persuasion tactics do scammers use, and can they be measured?	Sections 2.2.2 and 2.3.4
Methods for the component	Which models, transfer strategy, thresholds, OCR, explanations and evaluation practices suit the task?	Sections 2.2.3 to 2.2.8 and 2.3.3

2.2 Domain and Theoretical Foundations

2.2.1 Online recruitment fraud and its two phases

Online recruitment fraud (ORF) is the use of fake job advertisements, fake employers or fake recruiters to obtain money, personal data or labour from job seekers. Vidros et al. [5] described its main forms, such as advance-fee scams and identity harvesting, and released the Employment Scam Aegean Dataset (EMSCAD), which became the standard benchmark for detecting fraudulent advertisements. Most recruitment scams, however, are completed in a second phase that takes place outside the job platform. After the first contact, the "recruiter" moves the victim to SMS, WhatsApp, Telegram or email and asks for registration fees, training costs, document copies or bank details. Pitumpe and Rahmati [2] built an automated pipeline that collected more than 29,000 recruitment scam messages and engaged with more than 1,900 scammers, and found heavy reuse of message templates, domains and cryptocurrency wallets. Template reuse suggests that the language of scam conversations has regular patterns that a text classifier can learn.

2.2.2 Persuasion and manipulation tactics

Scam conversations rely on well-known principles of persuasion. Cialdini [6] described social proof, in which people follow what others appear to do or endorse,

and scarcity, in which an opportunity seems more valuable when it is limited or about to disappear. A related effect, often called the sunk-cost effect, leads people to keep investing in a course of action because of what they have already spent. Anagha et al. [1] applied these ideas to job scams, distinguishing payment-based scams that demand upfront fees from task-based scams that start with small rewards before escalating to financial demands. They operationalized urgency and time pressure, social proof, sunk-cost influence and fear of missing out (FOMO) as measurable signals, and found that urgency cues were significantly associated with payment, while FOMO cues could not be reliably identified under their operationalization. Their work is survey-based and does not provide an automated detector.

This dissertation uses the four tactics of Anagha et al. [1] with the following working definitions, which are used consistently in the labels, the model output and the warnings: urgency pressure creates artificial deadlines or time pressure; social proof uses fake authority or endorsements to appear legitimate; FOMO suggests that the opportunity will be lost to others or is limited; and sunk-cost influence uses money or effort already spent to demand further payment.

2.2.3 Transformer language models and DistilBERT

Transformer models use self-attention to model relationships between all the words in a text. BERT [7] pre-trains a transformer encoder on large text corpora and can then be fine-tuned for classification, and it captures context, paraphrase and word order better than keyword or bag-of-words models. DistilBERT [8] is a smaller version of BERT produced by knowledge distillation; its authors report that it is 40% smaller and 60% faster than BERT while retaining 97% of its language understanding capability. Because the BSTD component must classify chat messages in real time on a modest server, DistilBERT offers a practical balance between accuracy and latency.

2.2.4 Intermediate-task transfer

A pre-trained model can be fine-tuned first on an intermediate task and then on the target task. Pruksachatkun et al. [9] studied this intermediate-task transfer for pre-

trained language models and found that the benefit is greatest when the intermediate task is related to the target task. In this component, binary scam classification (Phase 1) is the intermediate task, and tactic classification (Phase 2) is the target task, which is closely related because every tactic is a property of scam language. Phase 2 therefore starts from the Phase 1 encoder instead of the general DistilBERT checkpoint.

2.2.5 Multi-label classification and decision thresholds

A single message can use several tactics at once, so tactic detection is a multi-label problem in which each message may receive any subset of the four labels. Multi-label classifiers are evaluated with measures such as Hamming loss, subset (exact-match) accuracy, and macro- and micro-averaged F1. Macro F1 weights each label equally and therefore exposes weak, rare labels, while micro F1 reflects overall performance across all decisions. A multi-label model outputs a probability for each label, and a fixed threshold of 0.5 is not necessarily best for every label, especially for rare labels, so tuning a separate threshold for each label on validation data is a common practice. This motivates the per-tactic thresholds of the BSTD component, which are chosen on the validation set.

2.2.6 Optical character recognition

Many job seekers receive scam messages on platforms that do not allow automated access, and the easiest evidence to share is a screenshot. Optical character recognition (OCR) converts the text in an image into machine-readable text. Tesseract is a widely used open-source OCR engine, and EasyOCR is a deep-learning-based open-source OCR library. Shinde et al. [4] showed that OCR can extract scam text from images so that it can be classified, which removes the need for platform API access.

2.2.7 Explainability

Transformer models do not explain their decisions on their own. SHAP [10] attributes a prediction to input features using Shapley values, and LIME [11] explains a single prediction by fitting a simple local model around it. For text, both methods can show which words pushed a message towards a scam decision. Uddin and Sarker [12]

combined a transformer phishing detector with LIME explanations, but the explanations were aimed at analysts. A job seeker needs a short, plain statement of what is wrong with the message, so in this component the word attributions are mapped to warning templates that name the tactic in everyday language.

2.2.8 Data sources, augmentation and leakage-aware evaluation

Real recruitment-scam conversations are scarce and private, so detectors are usually trained on a combination of sources. The UCI SMS Spam Collection [13] is a public corpus of labelled spam and legitimate SMS messages, and EMSCAD [5] provides fraudulent and genuine job advertisements. Back-translation, translating a sentence into another language and back, is a common way to create paraphrased training examples [14]. Combining sources and augmentation, however, creates a risk of leakage: when near-duplicate versions of the same message appear in both training and test data, the test score overstates performance on new messages. Kaufman et al. [15] described leakage as a frequent cause of over-optimistic results and recommended designing the data split to prevent it. Grouping related messages so that they fall into the same subset, and keeping test messages out of every training stage, are the standard remedies; Section 1.7.2 notes where the current evaluation falls short of this.

2.3 Related Research

2.3.1 Fake job advertisement detection

Most research on recruitment fraud classifies the job advertisement. Vidros et al. [5] introduced EMSCAD and baseline classifiers, and later work applied a range of machine learning models to the same dataset. Taneja et al. [16] proposed Fraud-BERT, a transformer-based, context-aware detector of fraudulent advertisements. These studies give strong advertisement-level detectors, but none of them analyses the messages exchanged after the first contact.

2.3.2 Scam and phishing message detection

Ghourabi [3] proposed SM-Detector, a BERT-based model for SMiShing messages on mobile devices, and showed that transformer models detect fraudulent SMS more reliably than keyword filters. Uddin and Sarker [12] fine-tuned a transformer for phishing email detection and explained its decisions with LIME. Tan et al. [17] proposed the Anticipate, Simulate, Reason (ASR) framework, in which a fine-tuned language model (ScamGPT-J) predicts scammer replies to give real-time warnings in instant messaging, using a dataset of 902 synthetic and seeded scam conversations; they reported that job scams benefited most. These studies show that conversation-level detection is feasible, but they produce a scam or risk score rather than the specific manipulation tactic, they mostly target a single channel, and their explanations, where present, are not designed for non-technical users.

2.3.3 OCR-based scam detection

Shinde et al. [4] developed an SMS scam detection application that extracts text from images with OCR and then classifies it. They compared unsupervised methods (K-means, non-negative matrix factorization and Gaussian mixture models) and deep semi-supervised models (RNN, LSTM and Bi-LSTM variants), with a KNN-Flatten model performing best at 94.13% accuracy. The work confirms the value of screenshot input, but the classifiers do not use pre-trained transformers and do not identify tactics.

2.3.4 Behavioural studies of job scams

Anagha et al. [1] provided the four-tactic view of job scams that this component adopts, and Pitumpe and Rahmati [2] provided large-scale evidence of how recruitment scam messages are structured and reused. Neither study turns its findings into an automated classifier that can warn a job seeker during a conversation. Table 2.2 compares the closest related studies with this research.

Table 2.2: Comparison of the closest related studies with this research

Study	Input	Method	Output	Tactics	User-level explanation
Vidros et al. [5]	Job advertisements	Classical ML	Fraud / genuine	No	No

Taneja et al. [16]	Job advertisements	Transformer (Fraud-BERT)	Fraud / genuine	No	No
Ghourabi [3]	SMS	BERT	Smishing / legitimate	No	No
Uddin and Sarker [12]	Email	Transformer + LIME	Phishing / legitimate	No	Analyst-oriented
Shinde et al. [4]	Images of SMS	OCR + ML / Bi-LSTM	Scam / legitimate	No	No
Tan et al. [17]	Instant messages	Fine-tuned LLM	Scam warning	No	Yes
Anagha et al. [1]	Survey responses	Behavioural analysis	Tactic associations	Yes (theory)	Not applicable
This research (BSTD)	Live chat, pasted text, screenshots	Two-phase DistilBERT + OCR	Three-tier verdict + tactics	Yes (four tactics)	Plain-language warning

2.4 Existing Systems, Products and Techniques

Several commercial products already warn users about scam messages. Google introduced Scam Detection in Google Messages in March 2025; an on-device model looks for suspicious patterns in conversations with non-contacts and shows a "likely scam" warning with options to dismiss, report or block, initially in English in the United States, the United Kingdom and Canada [18]. Truecaller launched an AI-based fraud protection feature for SMS in India in April 2023 that checks both the sender and the message content, covers messages that impersonate employers among other categories, shows a red warning and disables links [19]. Table 2.3 compares these products with the BSTD component.

Table 2.3: Existing products for detecting scam messages

Product	**Channel**	**Input**	**Output**	**Recruitment focus**	**Names the tactic**
Google Messages Scam Detection [18]	SMS, MMS, RCS in Google Messages	Incoming messages	Likely-scam warning	General	No
Truecaller fraud protection [19]	SMS (Android, India)	Incoming messages	Red warning, links disabled	Job offers among other categories	No

BSTD (this research)	Built-in chat and any external platform	Live chat, pasted text, screenshots	Three-tier verdict, tactics, warning	Yes	Yes

Each product protects one channel and gives a general scam verdict. Neither of them is designed around recruitment conversations, and neither tells the job seeker which manipulation tactic is being used. Both are limited to one platform or region; for example, Google Messages Scam Detection launched only in three countries [18]. A job seeker in Sri Lanka who receives a suspicious recruiter message on WhatsApp therefore has no tool that explains the manipulation in the conversation.

2.5 Gap Synthesis and Conceptual Framework

2.5.1 Synthesis of gaps

Table 2.4 links each gap found in the review to the part of this research that addresses it.

Table 2.4: Gaps identified in the literature and how this research addresses them

Gap	Evidence from the review	How this research addresses it
G1 Advertisement-only focus	Fake job research classifies advertisements only (Section 2.3.1)	Analyses recruiter-candidate messages after the first contact
G2 Binary output without tactics	Message detectors return scam or legitimate (Section 2.3.2); the tactic taxonomy remains theoretical (Section 2.3.4)	Multi-label classifier for four tactics, initialised from the Phase 1 encoder
G3 Single-channel input	Detectors and products are tied to one channel or platform (Sections 2.3.2 and 2.4)	Three input modes: built-in chat, pasted text and screenshots with OCR
G4 Explanations not for job seekers	Explanations are absent or analyst-oriented (Sections 2.2.7 and 2.3.2)	SHAP and LIME attributions mapped to plain-language warnings that name the tactic
G5 Uncertain cases forced into two classes	Products give a single scam warning (Section 2.4)	Three-tier verdict with a SUSPICIOUS band

G6 Isolation from job choice	Detectors do not inform which jobs are recommended (Sections 2.3 and 2.4)	Binary communication-scam indicator that the recommendation component is designed to accept

2.5.2 Conceptual framework

The conceptual framework in Figure 2.1 combines the foundations of Section 2.2 with the gaps in Table 2.4. Messages enter through the built-in chat, as pasted text, or as screenshots, which are converted to text by EasyOCR with a Tesseract fallback. Phase 1 gives a scam probability, and Phase 2, initialised from the Phase 1 encoder, detects the four tactics using per-tactic thresholds. The three-tier combination rule turns these outputs into a SCAM, SUSPICIOUS or LEGITIMATE verdict, and the explainability step converts word attributions into a plain-language warning. The verdict, tactics and warning are returned to the Detect section, the chat warns the job seeker when a recruiter message is classed as SCAM or SUSPICIOUS, and a binary communication-scam indicator is provided for the recommendation component. The bottom of the figure shows the offline model development and evaluation that produce the models and thresholds.

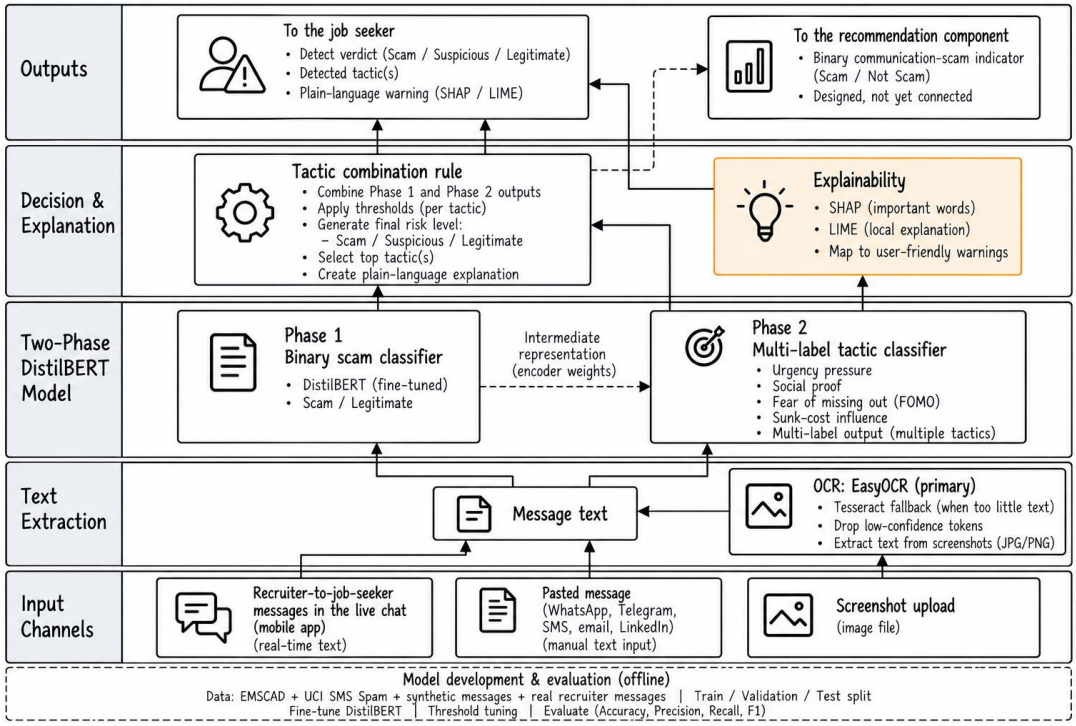


Figure 2.1: Conceptual framework of the BSTD component

3 RESEARCH METHODOLOGY

This chapter explains how the research was designed and carried out: the research design, the data, the experimental procedure, the measures, the analysis, the ethical and security considerations, and the artefacts that make the work reproducible. The design of the product itself is described in Chapter 4 and its implementation in Chapter 5.

3.1 Research Design

The research follows a quantitative design-and-evaluate approach. A solution, the BSTD component, was designed from the foundations in Chapter 2, and its central design decisions, namely the two-phase DistilBERT model and the per-tactic decision thresholds, were evaluated experimentally on labelled recruitment messages. The evaluation is a controlled offline experiment on held-out test data. No human participants took part in the experiment.

The design addresses the research questions of Section 1.6 as shown in Table 3.1. Choices that could affect the results, such as the training epoch and the tactic thresholds, were made on the validation split and judged on the test split, so that the reported performance is not an artefact of tuning.

Table 3.1: Research questions and the methods used to answer them

Research question	Method used	Section
RQ1 How can the datasets be built without leakage?	Collection and anonymisation of real messages; synthetic and back-translated data; labelling and verification; stratified split for Phase 1 and group-aware split for Phase 2	3.2
RQ2 How well does DistilBERT separate scam from legitimate messages?	Phase 1 fine-tuning on the training split; epoch chosen on validation data; accuracy and per-class metrics on the test split	3.3 and 3.4
RQ3 How accurately are the four tactics detected, and which are hardest?	Phase 2 intermediate-task transfer; per-tactic threshold search on validation data; per-tactic and multi-label metrics on the test split	3.3 to 3.5

RQ4 How can all input modes be handled and explained?	Pipeline design with OCR, a three-tier combination rule and SHAP and LIME warning templates	3.3, Chapters 4 and 5

3.2 Data, Participants or Study Materials

3.2.1 Participants and sources

The experiment used no human participants. The real messages were taken from public posts on Reddit, LinkedIn and Facebook in which job seekers shared recruitment messages they had received. In total, 718 real messages were collected. All of them were anonymised before use: personal names, phone numbers, email addresses and links were removed, and no attempt was made to contact the people who posted them. Handling followed the principles of Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [20].

3.2.2 Phase 1 dataset (binary)

The Phase 1 dataset contains 7,433 messages labelled scam or legitimate. It combines four sources: fraudulent and genuine job postings from EMSCAD [5], spam and legitimate messages from the UCI SMS Spam Collection [13], 1,000 synthetic recruitment messages, and the 718 real messages. The synthetic messages (650 scam and 350 legitimate) were written to imitate recruiter-candidate conversations on WhatsApp (283), email (267), LinkedIn (153), Telegram (128), SMS (74), Indeed (38), TopJobs.lk (35) and ikman.lk (22), and every synthetic message was reviewed manually. Table 3.2 summarizes the composition of the data.

3.2.3 Phase 2 dataset (tactics)

The Phase 2 dataset contains 3,840 messages, each labelled with any subset of the four tactics. It combines a manually curated tactic corpus, the reviewed synthetic messages, a tactic-pattern corpus, real messages and an email-scam corpus. Within the synthetic set, 383 messages carry urgency pressure, 268 social proof, 248 sunk-cost influence

and 227 FOMO, and 350 carry no tactic. To add realistic paraphrases, 105 real messages were back-translated as described in Section 3.2.5.

Table 3.2: Composition of the datasets

Item	Phase 1 (binary)	Phase 2 (tactics)
Total messages	7,433	3,840
Labels	Scam or legitimate	Urgency pressure, social proof, FOMO, sunk-cost influence (multi-label)
Sources	EMSCAD, UCI SMS Spam, 1,000 synthetic, 718 real	Curated tactic corpus, synthetic, tactic patterns, real messages, email-scam corpus, 217 back-translated copies
Split	Stratified 80/10/10, seed 42	Group-aware 70/10/20, seed 42
Test set	744 (491 legitimate, 253 scam)	758 (support: urgency 241, social proof 179, FOMO 101, sunk cost 68)

3.2.4 Labelling

The binary labels of the EMSCAD and UCI SMS Spam messages come from those datasets. The tactic labels were assigned by the author, using the working definitions of Section 2.2.2, and were then verified with an industry data scientist and with a large language model used as a second check. Inter-annotator agreement, for example with Cohen's kappa, was not computed, so the consistency of the tactic labels has not been measured formally; this is a limitation discussed in Chapter 7.

3.2.5 Back-translation

Back-translation [14] was applied to 105 real messages using the GoogleTranslator interface of the deep-translator Python library, with three pivot languages: English to Sinhala to English, English to Tamil to English, and English to French to English. Three quality filters were applied: the length of the copy had to be between 0.5 and 2.0 times that of the original, copies almost identical to the original (identity ratio of 0.95 or more) were removed, and copies with more than 85% word overlap with the original were dropped. Of 304 copies generated, 217 were kept (81 through Tamil, 69 through Sinhala and 67 through French). Back-translation was applied only to real messages, not to synthetic ones.

3.2.6 Data splits

The Phase 1 data were split at random into training (80%), validation (10%) and test (10%) sets, stratified by label, with the random seed fixed at 42. The test set contains 744 messages (491 legitimate and 253 scam). The Phase 2 data were split 70/10/20 with GroupShuffleSplit and seed 42, so that a message and its back-translated copies always fall in the same subset; the test set contains 758 messages.

Two weaknesses of these splits are stated openly. First, the Phase 1 split was neither deduplicated nor grouped, and Phase 2 validation and test messages were not excluded from Phase 1 training, so the shared encoder may already have seen some Phase 2 test messages. Second, the Phase 2 test set contains back-translated copies of some of its messages, which are near-duplicates. Both effects can make the reported results optimistic, as noted in Section 1.7.2; a retrained, leakage-free evaluation is planned.

3.3 Experimental or Study Procedure

The procedure has eight steps, shown in Figure 3.1.

1. Prepare the data: anonymise the real messages, add the synthetic messages, and generate and filter the back-translated copies.

2. Assign the four tactic labels and verify them (Section 3.2.4); the binary labels of EMSCAD and UCI SMS Spam come from those datasets.

3. Split the data as described in Section 3.2.6, with seed 42.

4. Train Phase 1: fine-tune distilbert-base-uncased with a binary classification head on the Phase 1 training split, using cross-entropy loss and the AdamW optimiser. The checkpoint with the best validation score was kept (epoch 3; validation F1 0.9175).

5. Train Phase 2: initialise the tactic classifier from the fine-tuned Phase 1 encoder (intermediate-task transfer), add a four-output sigmoid head, and train with binary

cross-entropy (BCEWithLogitsLoss) on the Phase 2 training split. The best checkpoint was kept (epoch 5; validation macro F1 0.8346).

6. Search thresholds: for each tactic separately, try thresholds from 0.25 to 0.60 in steps of 0.05 on the Phase 2 validation split, and choose the value with the highest validation F1 for that tactic. A tactic is flagged when its probability exceeds the threshold.

7. Freeze the models and thresholds.

8. Evaluate once on the test splits and report the metrics in Section 3.4.

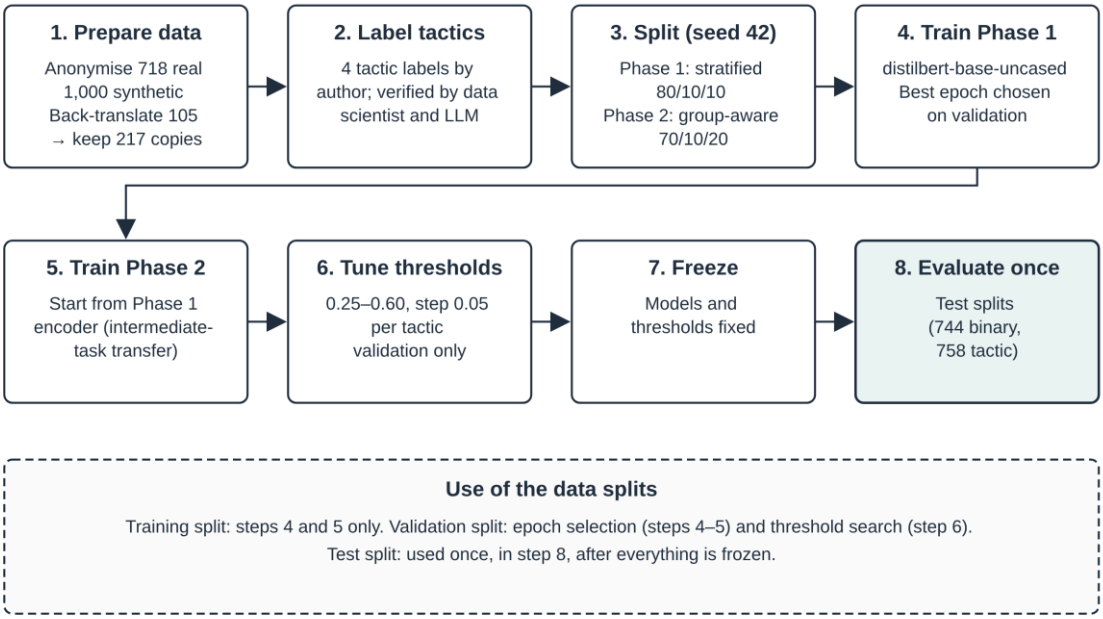


Figure 3.1: Research procedure from data to final evaluation

Table 3.3 lists the training settings and Table 3.4 the chosen thresholds. The deployed service uses the same thresholds. In the deployed pipeline, the outputs of the two models are combined by the three-tier rule of Section 1.2, and screenshots are converted to text by EasyOCR with a Tesseract fallback before classification; these steps are described in Chapters 4 and 5 and are not part of the metric evaluation.

Table 3.3: Training settings

Setting	Phase 1 (binary)	Phase 2 (tactics)
Starting model	distilbert-base-uncased	Fine-tuned Phase 1 encoder
Output head	2-class softmax	4 independent sigmoid outputs
Loss	Cross-entropy	BCEWithLogitsLoss
Learning rate	2e-5	2e-5
Epochs (best epoch)	3 (3)	6 (5)
Batch size	32	16
Maximum token length	128	128
Optimiser and schedule	AdamW, weight decay 0.01, linear schedule with 10% warm-up, gradient clipping 1.0	Same
Dropout	0.3	0.3
Random seed	42	42
Training time (Tesla T4)	4.9 minutes	5.3 minutes

Table 3.4: Per-tactic decision thresholds chosen on the validation split

Tactic	Threshold
Urgency pressure	0.25
FOMO	0.30
Sunk-cost influence	0.25
Social proof	0.45

A baseline model was also trained to check whether a fine-tuned transformer is needed for the binary task. It uses TF-IDF features with a logistic regression classifier with balanced class weights, and was trained on a separate set of 1,000 real recruitment messages (650 scam, 350 legitimate). It was checked with 5-fold cross-validation, its decision threshold (0.48) was chosen on a validation split, and it was evaluated once on an external test set of 374 real messages (278 scam, 96 legitimate). Its results are reported in Section 6.1.3.

3.4 Measures and Evaluation Metrics

For Phase 1, scam is treated as the positive class, and metrics are also reported for the legitimate class. For Phase 2, each tactic is evaluated as its own binary decision, and

the four decisions are also evaluated together as a multi-label prediction. Table 3.5 defines the measures.

Table 3.5: Evaluation measures

Measure	Definition	Used for
Accuracy	Share of messages classified correctly	Phase 1
Precision	Of the messages flagged with a label, the share that truly have it	Phase 1 per class; Phase 2 per tactic
Recall	Of the messages that truly have a label, the share that were flagged	Phase 1 per class; Phase 2 per tactic
F1-score	Harmonic mean of precision and recall	Phase 1 per class; Phase 2 per tactic
Macro F1	Mean of the per-tactic F1-scores, each tactic weighted equally	Phase 2
Micro F1	F1 computed over all tactic decisions pooled together	Phase 2
Hamming loss	Share of individual tactic decisions that are wrong	Phase 2
Exact-match accuracy	Share of messages whose full set of tactics is predicted exactly	Phase 2
Support	Number of test messages that truly have the label	Both phases

Macro F1 is the main measure for Phase 2 because it exposes weak, rare tactics such as FOMO, which micro F1 can hide. The hypotheses of Section 1.6.2 are judged against these measures: H1 uses the scam-class F1, H2 the macro F1, and H3 the per-tactic F1-scores.

3.5 Data Analysis

Predictions on the test splits were compared with the true labels, and the measures were computed with scikit-learn. Results are reported per class for Phase 1 and per tactic for Phase 2, together with the support of each label, so that the reader can see how many examples each figure rests on. Each hypothesis is marked as supported or not supported by comparing the measured value with its target.

A qualitative error analysis was also carried out. Misclassified test messages were read and grouped by the kind of mistake, for example scam messages that mix legitimate-sounding phrasing with subtle red flags, or genuine messages that mention other candidates. This analysis explains the numbers and informs the design of the three-tier combination rule. Confidence intervals and statistical significance tests were not computed; this is noted as a limitation in Chapter 7.

Table 3.6 links each objective of Section 1.5.2 to the measure that shows whether it was achieved and to the evidence reported later in this dissertation.

Table 3.6: Objectives, measures and evidence

Objective	Measure	Evidence
O1 Review related work and identify gaps	Gaps identified and linked to the design	Section 1.4, Chapter 2 (Table 2.4)
O2 Build the binary and tactic datasets	Dataset sizes, label distribution, split design	Section 3.2 (Table 3.2)
O3 Fine-tune DistilBERT for binary classification	Accuracy; scam-class F1 against H1 (at least 0.85)	Section 6.1
O4 Tactic classifier with per-tactic thresholds	Per-tactic F1; macro F1 against H2 (at least 0.80); tactic ranking against H3	Sections 3.3 and 6.1
O5 Three input modes and plain-language warnings	Working /classify, /classify-image and /ocr-only endpoints; example warnings	Chapters 4 and 5, Section 6.2
O6 Real-time service and integration	Recruiter chat messages checked and SCAM warnings shown in real time; indicator interface provided	Sections 5.3 and 6.4

3.6 Ethics, Privacy, Security and Responsible AI

Privacy. Only messages that had already been posted publicly were collected, and all 718 real messages were anonymised before labelling and training. No victims or senders were contacted, and no personal data are reported in this dissertation. The datasets and code are kept in private storage (OneDrive and a private GitHub repository) that only the project team can access.

Responsible use of the output. The component gives a risk estimate, not a legal judgement about any person or organisation. The API returns a separate SUSPICIOUS verdict so that uncertain messages are not presented as an accusation, and every warning states the reason in plain language so that the job seeker can judge it. The rule that suppresses a lone FOMO detection when Phase 1 is confident that a message is legitimate reduces false alarms on ordinary HR messages.

Bias and limits. The model covers English only and was trained partly on synthetic data, so it may perform worse on code-mixed Sinhala or Tamil messages and on writing styles not present in the data. These limits are stated in Section 1.7.2 and are shown to users through the wording of the warnings.

Security. The service exposes only the endpoints needed by the application, and messages sent for checking are used only to produce the result and the user's history.

Use of AI tools. A large language model was used as a second check during labelling, and an AI assistant was used for language editing and for checking analysis scripts, as disclosed in the declaration. All results were produced by the author's own code and verified by the author.

3.7 Reproducibility and Research Artefacts

Models were trained on Google Colab with an NVIDIA Tesla T4 GPU using Hugging Face Transformers version 5.15.1 and PyTorch. The Transformers version is recorded in the configuration files of the deployed models, which were saved by distilbert_final_model.ipynb; the experiment notebook did not record its version. The deployed services run on Python 3.12.4 (FastAPI 0.110.0, Uvicorn 0.29.0, PyTorch 2.13.0, Transformers 5.7.0) and Node.js 22.16.0. The source code is available at https://github.com/Kaju2002/R26-SE-002. The random seed was fixed at 42 for data splitting and training. The experiments reported in this dissertation were run in the notebook Distilbert_Scam_Detection_model.ipynb; the models used by the deployed

service were saved by the notebook distilbert_final_model.ipynb. The notebooks and datasets are stored in OneDrive, and the trained models and the service code are stored in the project's GitHub repository.

Table 3.7 lists the research artefacts.

Table 3.7: Research artefacts

Artefact	Content	Location
Distilbert_Scam_Detection_model.ipynb	Data preparation, splits, training of both phases, threshold search and the evaluation reported in Chapter 6	OneDrive
distilbert_final_model.ipynb	Training run that saved the models used by the deployed service	OneDrive
Datasets	Phase 1 (7,433 messages) and Phase 2 (3,840 messages) with labels and splits	OneDrive
Phase 1 and Phase 2 models	Saved DistilBERT models and configuration files	GitHub
Scam-detection service	FastAPI service, OCR pipeline, combination rule, warning templates and unit tests	GitHub

Two gaps limit exact reproduction. The notebooks install the latest library versions without pinning them, and the PyTorch version used on Colab was not recorded; the Transformers version also differs between training (5.15.1) and the runtime environment (5.7.0). Future runs will pin the versions, for example transformers==5.15.1, and record the full environment, so that the results can be reproduced exactly.

4 PRODUCT AND SOLUTION DESIGN

This chapter describes what the BSTD component must do, where it sits in the complete solution, which design alternatives were considered, how it is designed in detail, and which standards and professional practices it follows. Its implementation and testing are described in Chapter 5.

4.1 Requirements

4.1.1 Stakeholders and users

The primary users are job seekers, who receive recruiter messages in the built-in chat or on external platforms and want to know whether a message is a scam and why. Recruiters are secondary stakeholders: their chat messages are checked, so the component must avoid false alarms on genuine messages. Platform administrators deploy and configure the service. The fraud-aware job recommendation component is an internal consumer of the component's output.

4.1.2 Functional requirements

Table 4.1 lists the functional requirements. Each is traced to an objective of Section 1.5.2.

Table 4.1: Functional requirements

ID	Requirement	Objective
FR1	Accept a text message and return a scam decision (/classify)	O3, O6
FR2	Accept a screenshot, extract its text with EasyOCR and fall back to Tesseract when too little text is returned, then classify it (/classify-image)	O5
FR3	Return only the extracted text of a screenshot so that the user can check it (/ocr-only)	O5
FR4	Detect urgency pressure, social proof, FOMO and sunk-cost influence with a threshold for each tactic	O4
FR5	Combine both model outputs into a SCAM, SUSPICIOUS or LEGITIMATE verdict, including the lone-FOMO rule	O4, O6
FR6	Return word-level importance and a plain-language warning that names the detected tactics	O5

FR7	Check each text message sent by a recruiter to a job seeker in the built-in chat and warn the job seeker in real time when it is classed as SCAM or SUSPICIOUS	O6
FR8	Store each check and return the user's check history (/history)	O6
FR9	Provide a binary communication-scam indicator for the recommendation component	O6

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements

Quality	Requirement
Accuracy	Scam-class F1 of at least 0.85 and tactic macro F1 of at least 0.80 on held-out data (H1, H2)
Performance	/classify within 2 seconds for a typical chat message on CPU; /classify-image within 10 seconds
Explainability	Every SCAM or SUSPICIOUS verdict is accompanied by the detected tactics and a plain-language reason
Usability	Warnings are short, avoid technical terms and can be understood by non-technical job seekers
Privacy	Only the data needed for the result and the user's history are stored, in line with the PDPA [20]
Security	The service is reached through the API gateway, and request bodies are validated
Availability	The service is hosted in the cloud (Railway) so that the mobile and web clients can reach it
Maintainability	The component is a separate service with its own models, so it can be updated without changing the other services

For performance, the target is that /classify returns a result within 2 seconds for a typical chat message (up to 128 tokens) on CPU, so that the warning appears while the conversation is in progress. Screenshot checks (/classify-image) should finish within 10 seconds, because OCR runs first.

4.2 Component Architecture within the Complete Solution

Figure 4.1 shows the position of the BSTD component in the complete solution. The mobile application, built as an Android APK, and the web application, hosted on

Vercel, reach the back end through an API gateway. The back-end services are hosted on Railway and store their data in MongoDB. The BSTD service is a FastAPI application that exposes REST endpoints. Messages checked from the Detect section of the mobile application reach it through the gateway. Messages sent in the built-in chat are handled by the chat-management service, a Node.js service with Socket.IO, which calls POST /classify for each text message sent by a recruiter to a job seeker through utils/scamDetectionClient.js and, when the message is classed as SCAM or SUSPICIOUS, shows a warning to the job seeker in real time; messages are never blocked. The BSTD service stores each check in MongoDB so that the user's history can be returned. The binary communication-scam indicator is designed to be passed to the recommendation component, but this link is not yet connected. When a message is flagged, the chat service also publishes a chat.message.created event through RabbitMQ to the notification service. Table 4.3 summarizes the interfaces.

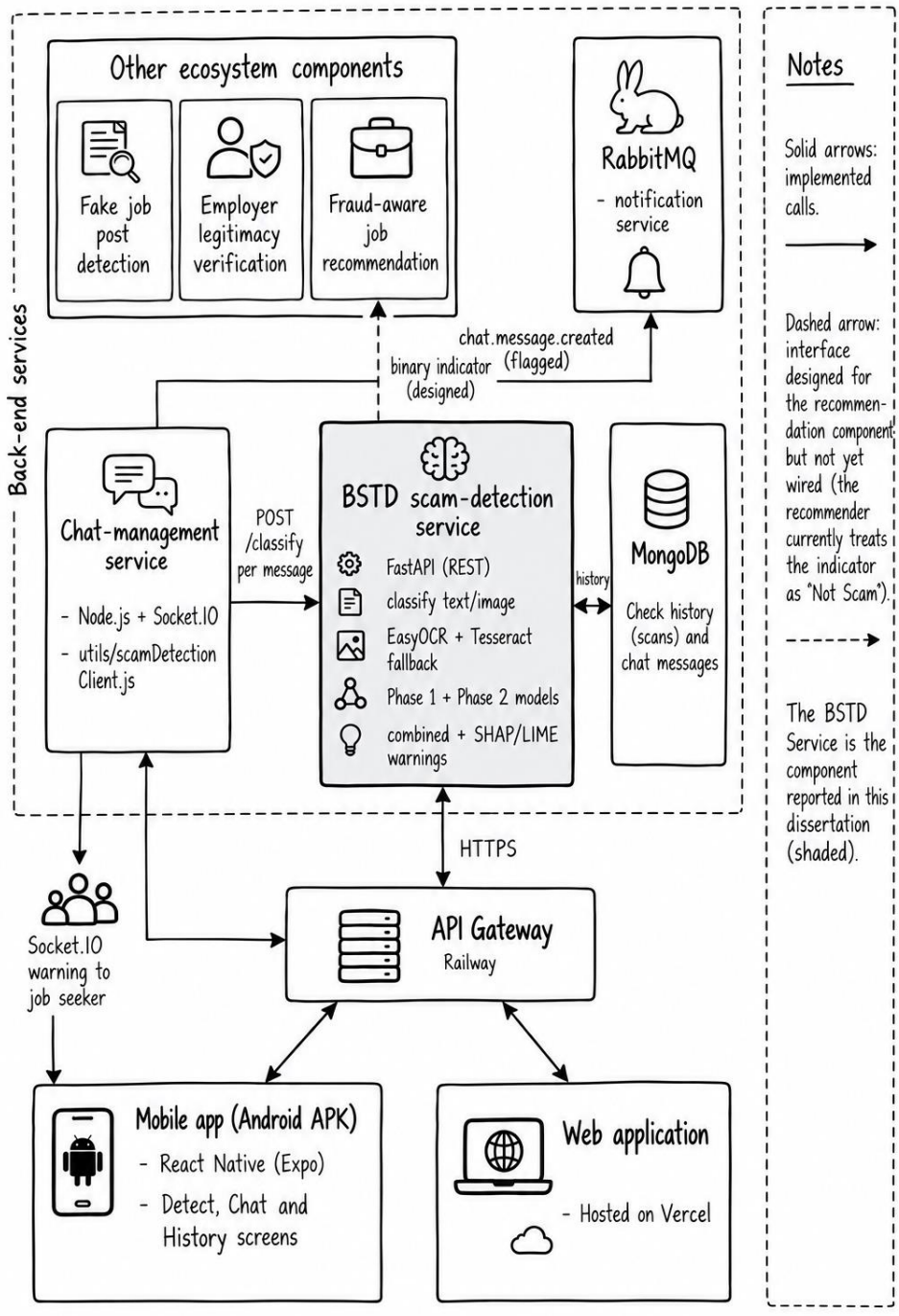


Figure 4.1: Position of the BSTD component in the complete solution

Table 4.3: Interfaces of the component

Interface	From / to	Content
POST /classify	Mobile app (via gateway) and chat-management service to BSTD	Message text in; verdict, tactics, confidence, word importance and warning out
POST /classify-image	Mobile app (via gateway) to BSTD	Screenshot in; extracted text and the same result as /classify out
POST /ocr-only	Mobile app (via gateway) to BSTD	Screenshot in; extracted text out
GET /history/{user_id}	Mobile app (via gateway) to BSTD	The user's previous checks
Socket.IO event	Chat-management service to mobile app	Result of the check pushed to the conversation
Database	BSTD to MongoDB	Stored checks for the history
Communication-scam indicator	BSTD to recommendation component (designed)	Scam / Not Scam flag; the recommender accepts an optional field but it is not yet filled

4.3 Design Alternatives and Decisions

Table 4.4 records the main design decisions, the alternatives that were considered and the reasons for each choice.

Table 4.4: Design decisions

Decision	Chosen	Alternatives considered	Reason
Language model	DistilBERT	BERT, RoBERTa; TF-IDF with logistic regression; a hosted large language model	Close to BERT in accuracy while smaller and faster [8]; runs on a modest server without per-request API cost
Training strategy	Two phases with intermediate-task transfer	Two independent models; one joint model	Tactic detection builds on what the encoder learned about scam language [9]
Tactic decisions	A threshold for each tactic	A fixed threshold of 0.5	Rare tactics need lower thresholds to be detected
Final output	Three-tier verdict	Binary scam or legitimate	Uncertain messages are shown as SUSPICIOUS instead of being forced into a class
OCR	EasyOCR with Tesseract fallback	Tesseract with OpenCV pre-processing	EasyOCR handles screenshot text well; the fallback recovers cases where EasyOCR returns

		(proposal); a cloud OCR API	too little text; no external data sharing
Explanation	SHAP and LIME mapped to warning templates	Raw scores; explanations generated by a large language model	Templates are predictable and understandable; attributions show the words behind the decision
API framework	FastAPI	Flask (proposal)	Built-in request validation, automatic OpenAPI documentation and asynchronous support
Real-time delivery	Socket.IO in the Node.js chat service	WebSockets inside the BSTD service	Keeps the model service stateless; the chat service already manages connections
Storage	MongoDB	A relational database	Check results are documents with varying fields such as tactics and word lists
Hosting	Railway (back end), Vercel (web)	Local server only	Reachable by the mobile and web clients during testing and demonstration

Table 4.5 lists the tools and **technologies used and why.**

Table 4.5: Tools and technologies

Tool or technology	**Use**
Python, PyTorch, Hugging Face Transformers	Model training and inference
scikit-learn	Data splitting and evaluation metrics
deep-translator	Back-translation of real messages
EasyOCR, Tesseract	Text extraction from screenshots
SHAP [10], LIME [11]	Word-level explanations
FastAPI	REST service for the component
Node.js, Socket.IO	Chat-management service and real-time delivery
MongoDB	Storage of check history
React Native (Expo)	Mobile application
Google Colab (Tesla T4)	Training environment
Railway, Vercel	Hosting of the back end and the web application
Git and GitHub	Version control

4.4 Detailed Design

4.4.1 Service interface

The BSTD service exposes eight REST endpoints, listed in Table 4.6. FastAPI generates the interactive documentation page shown in Figure 4.2 from the same definitions, and this page was used to test each endpoint during development.

Table 4.6: Endpoints of the BSTD service

Method and path	Purpose	Main input
GET /health	Reports whether the service and its models are ready	None
POST /classify	Classifies a text message and stores the result	text, user_id
POST /classify-image	Extracts the text of a screenshot with OCR, classifies it and stores the result	Image file, user_id
POST /ocr-only	Returns only the extracted text of a screenshot	Image file
GET /scan/{scan_id}	Returns the full result of one stored check	scan_id
GET /history/{user_id}	Returns the previous checks of a user	user_id
DELETE /history/clear/{user_id}	Deletes all stored checks of a user	user_id
DELETE /history/{scan_id}	Deletes one stored check	scan_id

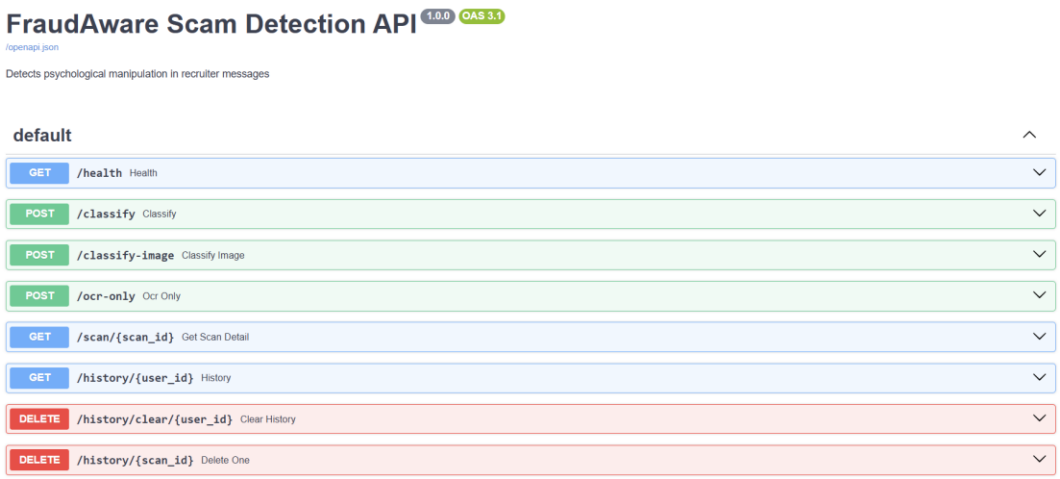


Figure 4.2: Endpoints of the BSTD service on the generated API documentation page

The /classify response contains scan_id, is_scam, inconclusive, confidence (an integer from 0 to 100), label, tactics, word_importance, warning, what_gave_it_away, original_text, extracted_text, source and created_at. Each detected tactic is returned with its name, key, probability and a short description, and word_importance lists the five most influential words. The responses for a scam message and a legitimate message are shown below; the user identifier in the requests is replaced and the original_text field, which repeats the request text, is shortened.

```
Request: POST /classify
{
  "text": "Congratulations! You have been selected for the remote data entry
           job with a salary of LKR 85,000 per month. To confirm your position,
           please pay the registration fee of LKR 5,000 today. Only a few seats
           are left and many candidates have already joined. If payment is not
           received within 24 hours, your offer will be cancelled.",
  "user_id": "<user identifier>"
}

Response: 200 OK
{
  "scan_id": "6acb7be584ca97974c7d34c0",
  "is_scam": true,
  "inconclusive": false,
  "confidence": 100,
  "label": "SCAM DETECTED",
  "tactics": [
    { "name": "FOMO", "key": "fomo", "score": 0.998,
      "description": "Implies other candidates will take your
                      opportunity if you delay" }
  ],
  "word_importance": [
    { "word": "seats", "score": 1 },
    { "word": "today", "score": 0.915 },
    { "word": "left", "score": 0.679 },
    { "word": "registration", "score": 0.649 },
    { "word": "are", "score": 0.6 }
  ],
  "warning": "Real companies do not pressure candidates by mentioning
              other applicants.",
  "what_gave_it_away": "The words seats, today, left triggered the FOMO
      detection. These patterns are commonly found in recruitment scam
      messages.",
  "original_text": "Congratulations! You have been selected for ...",
  "extracted_text": null,
  "source": "text",
  "created_at": "2026-10-11T12:07:01.507646+00:00"
}
```

The message is classed as SCAM. FOMO is detected with a probability of 0.998, and because the Phase 1 probability is high, the lone-FOMO rule does not apply. The words "seats", "today" and "left" have the highest importance. Although the message also

contains deadline wording ("today", "within 24 hours"), urgency pressure was not flagged, which is an example of the tactic-level errors discussed in Section 6.1.2.

```
Request: POST /classify
{
  "text": "Hi Kajan, thank you for applying for the Junior Software Engineer
           position at our company. We have reviewed your application and
           would like to invite you for a technical interview on Monday,
           19 October at 10:00 AM at our Colombo office. The interview will
           take about one hour. Kindly reply to confirm whether this time
           works for you, or suggest another time that suits you. Best
           regards, Nimali Perera, HR Executive",
  "user_id": "<user identifier>"
}

Response: 200 OK
{
  "scan_id": "6acb859984ca97974c7d34c1",
  "is_scam": false,
  "inconclusive": false,
  "confidence": 97,
  "label": "LEGITIMATE",
  "tactics": [],
  "word_importance": [
    { "word": "hi", "score": 1 },
    { "word": "thank", "score": 0.445 },
    { "word": "kajan", "score": 0.302 },
    { "word": "kindly", "score": 0.211 },
    { "word": "you", "score": 0.142 }
  ],
  "warning": "This message appears safe. No manipulation tactics were
             detected.",
  "what_gave_it_away": "No manipulation tactics were detected in this
      message.",
  "original_text": "Hi Kajan, thank you for applying for ...",
  "extracted_text": null,
  "source": "text",
  "created_at": "2026-10-11T12:48:25.178961+00:00"
}
```

The genuine interview invitation is classed as LEGITIMATE with a confidence of 97 and no tactic. Its most influential words are ordinary greeting and politeness words, and the warning tells the user that no manipulation tactic was found.

4.4.2 Processing sequence

A request to /classify or /classify-image is processed in the following order.

1. The request is validated. For /classify-image, the image is passed to EasyOCR, tokens with confidence below 0.12 are dropped, and if the cleaned text is shorter than 18 characters or EasyOCR fails, Tesseract is run and the longer result is kept.

2. The text is tokenized with the DistilBERT tokenizer and truncated to 128 tokens.

3. Phase 1 returns the scam probability p.

4. Phase 2 returns a probability for each tactic, and each is compared with its threshold (Table 3.4).

5. The combination rule (Section 4.4.3) produces the verdict and the decision stage.

6. Word importance is computed and mapped to a plain-language warning.

7. The result is stored in MongoDB for the history and returned to the caller.

4.4.3 Combination rule

The function combine() turns the Phase 1 probability and the detected tactics into the final verdict. Table 4.7 lists the rules in the order in which they are applied. Each rule is covered by a unit test.

Table 4.7: Rules of the combination function

Order	Condition	Verdict	Decision stage
1	FOMO is the only tactic detected and $p < 0.40$	LEGITIMATE	ensemble_legit
2	Any tactic detected	SCAM	phase2_tactic
3	No tactic and $p \geq 0.75$	SCAM	phase1_high
4	No tactic and $0.40 \leq p < 0.75$	SUSPICIOUS	phase1_uncertain
5	No tactic and $p < 0.40$	LEGITIMATE	ensemble_legit
6	Phase 1 unavailable and no tactic	LEGITIMATE	phase2_no_tactics

4.4.4 Explanation and warnings

Word-level attributions identify the words that pushed the message towards a scam decision. They are returned in the word_importance field and summarized in what_gave_it_away, and the detected tactics are mapped to warning templates written in plain language.

Each detected tactic is mapped to a fixed warning, and Table 4.8 lists the warning texts returned by the deployed service. For example, the message "You have already paid the LKR 3,000 registration fee and completed the training, so you are almost done. To release your salary, please pay the final processing fee of LKR 7,500. Otherwise all the money and effort you invested will be lost" was classed as SCAM with sunk-cost

influence at a probability of 0.997, and the words "please", "otherwise" and "lost" were the most influential.

Table 4.8: Warning texts shown to the user

Situation	Warning text
Urgency pressure detected	Legitimate employers never impose same-day deadlines or payment cutoffs.
FOMO detected	Real companies do not pressure candidates by mentioning other applicants.
Social proof detected	Always verify company credentials independently before proceeding.
Sunk-cost influence detected	Do not feel obligated to continue because of time or money already spent.
SCAM from the Phase 1 score, no tactic detected	This message matches recruitment fraud patterns (payment requests, personal data, or credential harvest) even without obvious pressure tactics.
No tactic detected (legitimate)	This message appears safe. No manipulation tactics were detected.
All results (shown in the application)	Estimates from a model — always verify the employer yourself before sharing personal or financial details.

The word_importance scores come from the SHAP and LIME attributions described in Section 2.2.7, and the highest-scoring words are named in what_gave_it_away.

4.4.5 Chat integration

In the built-in chat, the chat-management service sends each text message from a recruiter to a job seeker to POST /classify through utils/scamDetectionClient.js, and the outcome is delivered to the job seeker through Socket.IO.

Only text messages sent by recruiters to job seekers are checked, and only the first 2,000 characters are sent. Flagged messages are never blocked. The job seeker sees a red strip inside the message bubble that reads "This message looks unsafe. Don't send money or personal details.", with up to two tactic labels and a report button; if the job seeker is elsewhere in the application, an in-app banner titled "Possible scam message" appears. The recruiter sees no warning. Details are given in Section 5.3.

Figure 6.3 shows the warning as it appears to the job seeker.

4.4.6 Data storage

Each check is stored as one document in the scans collection of the fraudaware MongoDB database. GET /history returns the documents of a user, GET /scan/{scan_id} returns one document, and the two DELETE endpoints let the user remove one check or the whole history. Table 4.9 lists the fields of a document, and Figure 4.3 shows a stored check of a screenshot, with the user identifier and the message text hidden.

Table 4.9: Fields of a document in the scans collection

Field	Content
_id	Identifier of the check, returned to the client as scan_id
user_id	Identifier of the user who made the check
original_text	The message text as submitted, or the text extracted from the screenshot
is_scam	Whether the message was classed as a scam
confidence	Confidence of the decision, from 0 to 100
tactics	The detected tactics with their probabilities
word_importance	The words with the highest importance and their scores
warning	The plain-language warning shown to the user
what_gave_it_away	A short explanation naming the words and the tactic behind the decision
source	text or image
extracted_text	The OCR output for a screenshot; empty for a text message
created_at	Date and time of the check (UTC)



Figure 4.3: A stored check in the scans collection (personal data hidden)

4.4.7 Code structure

The service is kept in the services/scam-detection folder of the project repository. Figure 4.4 shows its structure and Table 4.10 describes the role of each main file and folder.

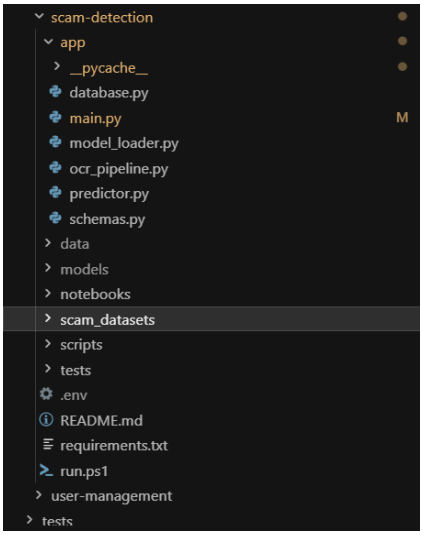


Figure 4.4: Folder structure of the scam-detection service

Table 4.10: Main files and folders of the scam-detection service

File or folder	Role
app/main.py	FastAPI application: defines the endpoints and loads the models at start-up
app/model_loader.py	Loads the Phase 1 and Phase 2 DistilBERT models, the tokenizer and the thresholds
app/predictor.py	Runs both models, applies the combination rule and builds the explanation and warning
app/ocr_pipeline.py	EasyOCR text extraction with the Tesseract fallback
app/database.py	Connection to MongoDB and the functions that store, read and delete checks
app/schemas.py	Request and response models used for validation
models/	The trained model files
data/, scam_datasets/	Datasets used for training and evaluation
notebooks/	Training and evaluation notebooks
scripts/	Helper scripts for data preparation
tests/	Tests of the service
requirements.txt	Python dependencies
run.ps1	Script that starts the service during development

.env	Settings such as the database connection (not stored in the repository)

4.5 Standards and Professional Practice

4.5.1 Interface standards

The service follows REST conventions and exchanges JSON over HTTPS. FastAPI generates an OpenAPI description of every endpoint, which serves as the interface contract for the mobile application and the chat-management service.

4.5.2 Data protection

The design follows the principles of Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [20]: only the data needed to produce a result and the user's history are stored, the training data were anonymised, and the checked messages are stored only in the project's MongoDB database.

4.5.3 Professional ethics

The work follows the principles of professional computing ethics, in particular the duties to avoid harm, to be honest about the limits of a system and to respect privacy. The component is presented as a risk estimate, its limitations are stated openly, and its warnings explain the reason for each decision.

4.5.4 Engineering practice

The code is kept under version control in GitHub. The component is a separate service with a single responsibility, so that it can be updated, tested and deployed independently. The combination rule is covered by unit tests that run without model weights, and the experiments are kept in notebooks so that they can be repeated.

The chat-management service uses JWT authentication and an internal service key. The BSTD service currently allows cross-origin requests from any origin; restricting this is listed as a finding in Section 5.4.4.

5 IMPLEMENTATION, INTEGRATION AND TESTING

This chapter describes how the design of Chapter 4 was built, how the component works with the other parts of the system, how it was verified, and how it is deployed and operated.

5.1 Development Environment and Technology Stack

The models were trained on Google Colab, and the services run in the deployed runtime environment. Because the two environments use different library versions, they are listed separately in Table 5.1. The source code is kept in the repository https://github.com/Kaju2002/R26-SE-002.

Table 5.1: Development environment

Item	Training (Google Colab)	Runtime environment (services)
Hardware	NVIDIA Tesla T4 GPU	Railway; CUDA used if available, otherwise CPU
Language and runtime	Python (Colab default)	Python 3.12.4; Node.js 22.16.0 with npm 11.6.4
Deep learning	PyTorch (version not recorded); Transformers 5.15.1 (recorded by the deployed models)	PyTorch 2.13.0; Transformers 5.7.0
Service framework	Not applicable	FastAPI 0.110.0 with Uvicorn 0.29.0; Pydantic request models
OCR	Not applicable	EasyOCR 1.7.2; Tesseract (fallback)
Database	Not applicable	MongoDB Atlas through PyMongo 4.17.0
Other libraries	pandas, NumPy, scikit-learn, deep-translator	Pillow, OpenCV, python-multipart, python-dotenv, Motor
Messaging	Not applicable	Socket.IO (chat), RabbitMQ (events)
Code repository	OneDrive (notebooks)	GitHub: Kaju2002/R26-SE-002

The requirements.txt file of the service lists its packages without version numbers, so the versions in Table 5.1 were read from the environments that were used. Pinning these versions is part of the future work (Section 5.5).

5.2 Component Implementation

The design of the service, the combination rule and the warnings are described in Chapter 4. This section describes how they were built and the settings and limits that affect their behaviour.

5.2.1 Start-up and model loading

The service is a FastAPI application. When it starts, a start-up handler connects to MongoDB and then loads the Phase 2 model followed by the Phase 1 model, once, so that they are not reloaded for each request. Requests that arrive before loading has finished receive the response 503 "Model not loaded". The models run on the GPU when CUDA is available and on the CPU otherwise. EasyOCR is loaded lazily on the first image request and then reused, so the first screenshot check takes noticeably longer than later ones.

5.2.2 Handling a request

A request is processed in the order given in Section 4.4.2. The text is truncated to 128 tokens, both models are applied, the combination rule produces the verdict, the warning is built, and the result is saved as one document in the scans collection of the fraudaware database before it is returned.

5.2.3 Settings and limits

Table 5.2 lists the main settings of the two services involved, with secret values removed, and Table 5.3 lists the time and size limits.

Table 5.2: Main settings of the services (secrets removed)

Service	Setting	Default or value	Purpose
BSTD (FastAPI)	MONGO_URI	Required	MongoDB Atlas connection (database fraudaware, collection scans)
BSTD (FastAPI)	MODEL_PATH	./models/phase2_distilbert	Phase 2 tactic model
BSTD (FastAPI)	PHASE1_PATH	./models/phase1_distilbert	Phase 1 binary model

BSTD (FastAPI)	TESSERACT_PATH	Tesseract installation path	Tesseract fallback for OCR
BSTD (FastAPI)	EASYOCR_GPU	Off	Lets EasyOCR use the GPU
BSTD (FastAPI)	Port	8000	Set in the Uvicorn command
BSTD (FastAPI)	CORS	All origins; GET, POST, DELETE	Cross-origin access
Chat (Node.js)	PORT	5003	Chat-management service port
Chat (Node.js)	SCAM_DETECTION_BASE_URL	URL of the BSTD service	Address of the BSTD service
Chat (Node.js)	MONGO_URI	Chat database (fraudaware-chat)	Chat storage
Chat (Node.js)	RABBITMQ_URL	RabbitMQ connection (password removed)	Event publishing
Chat (Node.js)	JWT_SECRET, INTERNAL_SERVICE_KEY, Cloudinary keys	Secret (not shown)	Authentication and media

Table 5.3: Time and size limits

Where	**Limit**
Chat service calling /classify	15 seconds; on time-out or error, the scan is saved as an error and the message is still delivered
Text sent for checking from chat	First 2,000 characters
Model input	128 tokens; longer text is truncated
FastAPI request time-out	None set
MongoDB	Library defaults; no time-out configured
In-app warning banner	Disappears after 6.5 seconds

5.2.4 Rules written in the code

The per-tactic thresholds and the combination rule are written in predictor.py. The combination rule matches Table 4.7 and is covered by unit tests. The thresholds are those selected on the validation split (Table 3.4): 0.25 for urgency pressure, 0.30 for FOMO, 0.25 for sunk-cost influence and 0.45 for social proof, and a tactic is flagged when its probability exceeds its threshold, as in the experiment. An earlier version of

the code used different values; this was found during code review and corrected (Section 5.4.4).

5.3 Integration with the Other Components

5.3.1 Mobile application

The Detect section of the mobile application sends pasted text to /classify, screenshots to /classify-image or /ocr-only, and reads previous checks from /history, through the API gateway.

Figure 5.1 shows the Detect section, where the user can paste a message of up to 2,000 characters, upload a screenshot or upload several screenshots of a conversation, and the list of recent checks, where each check can be opened or deleted. Results of individual checks are shown in Section 6.4.

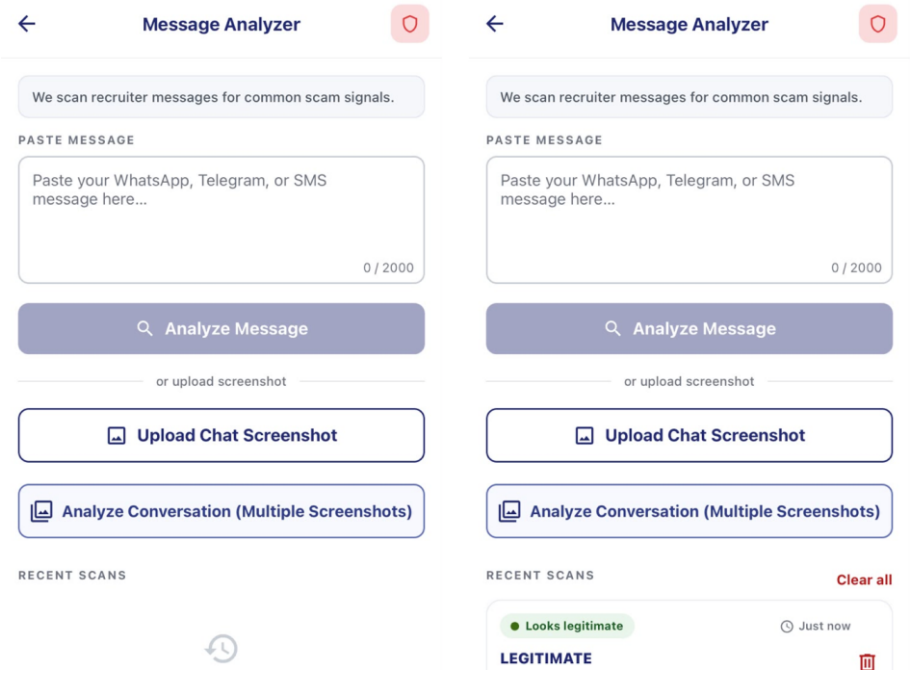


Figure 5.1: Detect section of the mobile application and the list of recent checks

5.3.2 Built-in chat

The chat-management service checks only text messages sent by recruiters to job seekers; images, voice notes and files are not checked. For each message, utils/scamDetectionClient.js sends the first 2,000 characters to POST /classify with a time-out of 15 seconds. Messages are never blocked. When the result has is_scam set to true, the job seeker sees a red strip inside the message bubble with the text "This message looks unsafe. Don't send money or personal details.", up to two tactic labels, and a report button. If the job seeker is in another part of the application, an in-app banner titled "Possible scam message — From {sender}" appears. The chat.message.created event also carries flagged: true to the notification service through RabbitMQ. The recruiter does not see any warning. A SUSPICIOUS result is shown to the job seeker with a softer caution, so that uncertain messages are marked without being presented as scams.

Figure 6.3 shows a flagged message in the chat and the matching entry in the notification list.

5.3.3 Recommendation component

The recommendation component accepts an optional boolean field, commIsScam, which indicator_mapper.py maps to "Scam" or "Not Scam". The mobile application passes job.commIsScam, but no part of the system fills it from the BSTD results yet, so the recommender currently treats every job as "Not Scam". The component therefore provides the indicator interface, and connecting it is future work.

5.4 Verification and Software Testing

5.4.1 Test approach

Two kinds of tests were used. Automated tests check the endpoints, the combination rule, the explanations and the thresholds; most of them run without loading the models, so they run quickly. Scenario tests send real requests to the deployed service and check

the behaviour of the whole pipeline, including the models, OCR and storage. The automated tests are kept in the tests/scam_detection folder of the repository and are run with py -m pytest tests/scam_detection -v.

5.4.2 Automated tests

The automated tests were run with py -m pytest tests/scam_detection -v. Of the 20 tests, 18 passed and 2 were skipped, in 39.51 seconds. Table 5.4 lists the tests and the requirement that each verifies, and Figure 5.2 shows the output of the run. The two integration tests in test_scam_predict_integration.py were skipped in this run; integration testing of the full prediction pipeline is still in progress.

Table 5.4: Automated tests of the service

Test	**What it checks**	**Requirement**	**Result**
api/test_classify_api.py			
test_health_ok	The health endpoint reports that the service is ready	Availability	Passed
test_classify_success	A valid request to /classify returns a complete result	FR1, FR8	Passed
test_classify_rejects_empty_text	An empty message is rejected	FR1	Passed
test_classify_rejects_missing_user_id	A request without a user identifier is rejected	FR8	Passed
unit/test_combine.py			
test_combine_rule1_tactics_means_scam	Any detected tactic gives SCAM, even with a low p	FR5	Passed
test_combine_rule2_confident_legit	No tactic and p below	FR5	Passed

	0.40 gives LEGITIMATE		
test_combine_rule3_no_phase1_still_legit	Without a Phase 1 score and no tactic, LEGITIMATE	FR5	Passed
test_combine_phase1_high_no_tactics_means_scam	p of 0.75 or more with no tactic gives SCAM	FR5	Passed
test_combine_phase1_mid_band_is_suspicious	p between 0.40 and 0.75 gives SUSPICIOUS	FR5	Passed
test_combine_lone_fomo_suppressed_when_phase1_confident_legit	A lone FOMO with p below 0.40 gives LEGITIMATE	FR5	Passed
test_combine_response_shape	All response fields are present and confidence is an integer from 0 to 100	FR1, FR6	Passed
unit/test_explanations.py			
test_build_explanation_no_tactics	Explanation when no tactic is detected	FR6	Passed
test_build_explanation_with_tactics	Explanation names the detected tactics	FR6	Passed
test_build_warning_no_tactics	Warning when no tactic is detected	FR6	Passed
test_build_warning_with_tactics	Warning for detected tactics	FR6	Passed
unit/test_thresholds.py			

test_all_tactics_have_metadata	Every tactic has a name and description	FR4, FR6	Passed
test_thresholds_are_probabilities	Every threshold is between 0 and 1	FR4	Passed
test_phase1_band_ordering	The Phase 1 bands are in the correct order (0.40 below 0.75)	FR5	Passed
unit/test_scam_predict_integration.py			
test_predict_scam_fixture_flagged	A known scam message is flagged by the full pipeline	FR1, FR5	Skipped
test_predict_safe_fixture_shape	A safe message returns a complete response	FR1	Skipped

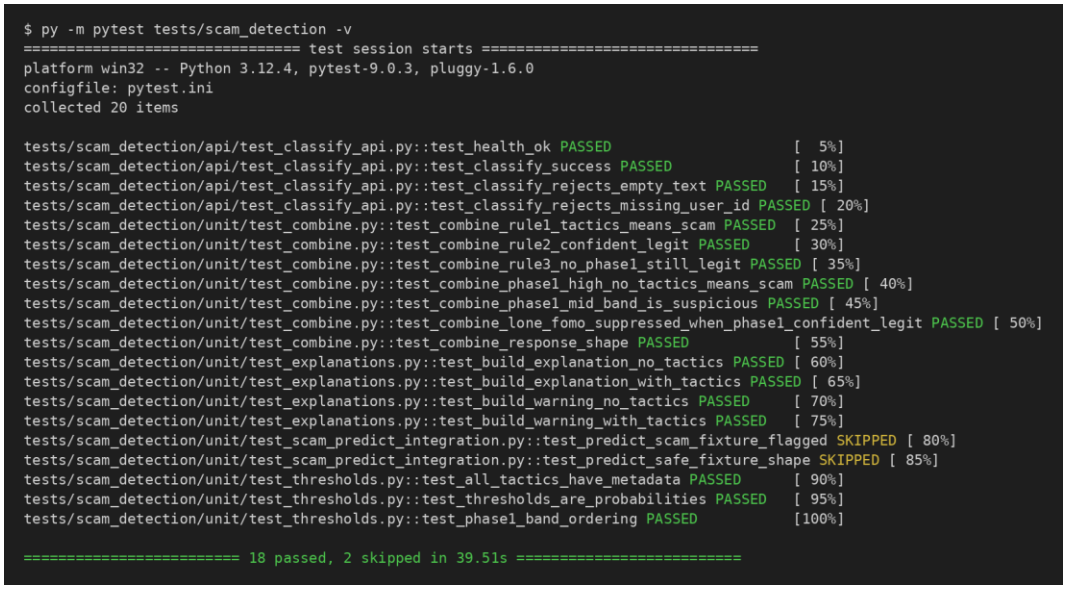


Figure 5.2: Output of the automated test run (18 passed, 2 skipped)

5.4.3 Scenario tests

The behaviour of the whole service was tested with the scenarios in Table 5.5. Scenarios 1 to 3 were run from the Detect section of the mobile application against the deployed service, and their results are shown in Figures 6.4 and 6.5; scenario 4 is covered by an automated test

Table 5.5: Scenario tests of the service

No.	Input	Expected result	Actual result
1	Upfront-fee request ("Pay a 5000 LKR registration fee today to confirm your position…")	SCAM, with urgency or sunk-cost influence	SCAM; urgency pressure detected (Figure 6.4)
2	Genuine HR message following up after an interview	LEGITIMATE	LEGITIMATE; no tactic detected (Figure 6.4)
3	Screenshot of a scam message	Text extracted, then SCAM	Text extracted; SCAM from the Phase 1 score, no tactic flagged (Figure 6.5)
4	Empty message	Rejected	Rejected (automated test test_classify_rejects_empty_text)

5.4.4 Findings from testing and code review

Reviewing the code alongside the tests identified the following issues. The first three were corrected; the others remain open and are recorded here so that the behaviour of the service can be read correctly.

1. Thresholds differed (corrected). The code used thresholds and a comparison that differed from those selected on the validation split. The code was changed to the values of Table 3.4 and to the "exceeds" comparison, so the deployed service now behaves as evaluated in Chapter 6.

2. SUSPICIOUS was silent in the chat (corrected). The chat originally warned only when is_scam was true, so a SUSPICIOUS verdict produced no warning. A softer caution is now shown for SUSPICIOUS messages.

3. Empty messages (corrected). An empty message is now rejected by the request validation instead of being classified and stored, and this is checked by test_classify_rejects_empty_text.

4. Images in the chat are not checked. A scam sent as an image in the chat receives no warning, although the Detect section can check screenshots.

5. Security settings are permissive. The BSTD service allows cross-origin requests from any origin, and the MongoDB connection accepts invalid TLS certificates (tlsAllowInvalidCertificates=True).

6. A deprecated start-up hook is used. The @app.on_event("startup") handler is deprecated in current FastAPI versions; this does not affect behaviour.

5.5 Deployment and Operation

The back-end services are deployed on Railway. The BSTD service is started with Uvicorn on port 8000, the chat-management service runs on port 5003, and RabbitMQ carries the chat events. The web application is hosted on Vercel, and the mobile application is distributed as an Android APK. The services are deployed without Dockerfiles, so they are not containerised.

The BSTD service is started with the following command:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Four points matter for operation. First, the models take time to load, and requests sent during start-up receive a 503 response. Second, the first screenshot check is slow because EasyOCR is loaded on first use. Third, the Tesseract path must be set through TESSERACT_PATH to match the host on which the service runs. Fourth, the packages in requirements.txt are not pinned; pinning the versions in Table 5.1 (for example fastapi==0.110.0) is needed for a reproducible deployment.

6 RESULTS AND VALIDATION

This chapter reports what the experiments and tests showed, in the order of the research questions of Section 1.6: the results of the two models on held-out data, the technical results of the service, stakeholder validation, and the results of integration. A summary of the findings closes the chapter. The meaning of the results is discussed in Chapter 7.

6.1 Research Results

6.1.1 Phase 1: binary scam classification

The Phase 1 model reached its best validation F1-score of 0.9175 at epoch 3. On the held-out test split of 744 messages (491 legitimate and 253 scam), it reached an accuracy of 93.82%, with F1-scores of 0.9538 for the legitimate class and 0.9065 for the scam class (Table 6.1 and Figure 6.1). Precision is high for both classes (0.94 for legitimate and 0.93 for scam messages), so few legitimate messages are flagged: 16 of the 491 legitimate messages were classed as scams. Scam recall is lower (0.88): 223 of the 253 scam messages were detected and 30 were missed, so about one scam message in eight passes as legitimate. The test F1 for the scam class (0.9065) is close to the validation F1 (0.9175), which suggests that the model was not over-fitted to the validation split.

Table 6.1: Phase 1 results on the test split

Class	Precision	Recall	F1-score	Support
Legitimate	0.9406	0.9674	0.9538	491
Scam	0.9331	0.8814	0.9065	253
Accuracy			93.82%	744
Macro F1			0.9302	

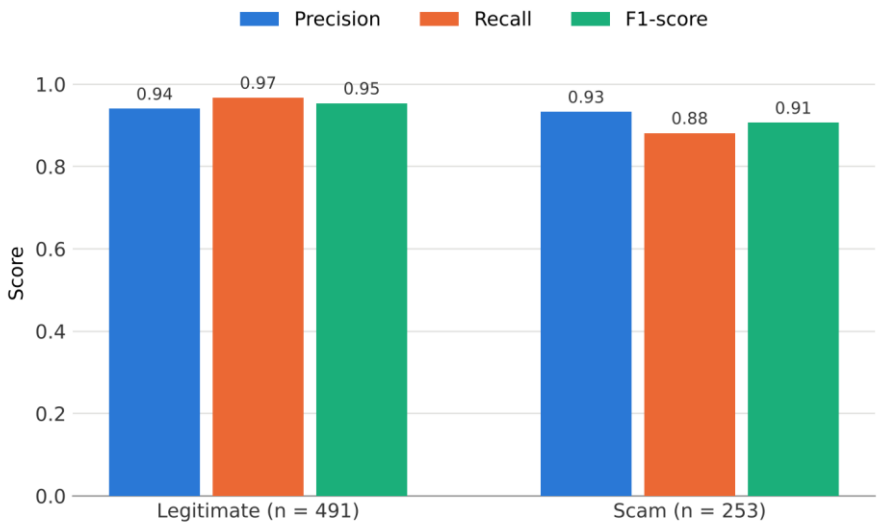


Figure 6.1: Phase 1 precision, recall and F1-score by class on the test split

6.1.2 Phase 2: tactic classification

The Phase 2 model reached its best validation macro F1 of 0.8346 at epoch 5. On the held-out test split of 758 messages, it reached a macro F1 of 0.8345 and a micro F1 of 0.8722, with an exact-match accuracy of 85.36% and a Hamming loss of 0.0488 (Table 6.2 and Figure 6.2). The validation and test macro F1-scores are almost identical, which suggests that the per-tactic thresholds chosen on the validation split were not over-fitted to it.

Table 6.2: Phase 2 results on the test split

Tactic	Precision	Recall	F1-score	Support
Urgency pressure	0.9055	0.9544	0.9293	241
Social proof	0.9153	0.9050	0.9101	179
Sunk-cost influence	0.8657	0.8529	0.8593	68
FOMO	0.7746	0.5446	0.6395	101
Macro F1			0.8345	
Micro F1			0.8722	
Exact-match accuracy			85.36%	758
Hamming loss			0.0488	

Urgency pressure and social proof were detected most reliably, with F1-scores above 0.90. Both have many training examples and consistent wording, such as deadlines and references to other applicants or to authority. Sunk-cost influence reached an F1 of 0.86 despite having the smallest test support (68 messages). FOMO was the weakest tactic (F1 0.6395). Its precision of 0.77 shows that the model is usually right when it flags FOMO, but its recall of 0.54 shows that it misses almost half of the FOMO messages. FOMO is expressed in more varied language than urgency, and it overlaps with urgency in many messages. This agrees with Anagha et al. [1], who could not reliably identify FOMO cues under their operationalization.

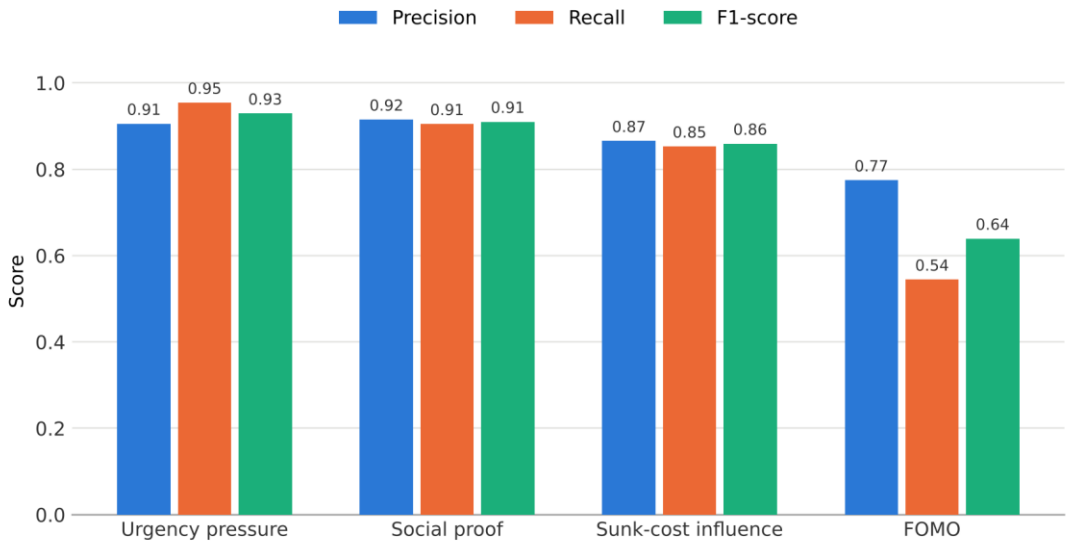


Figure 6.2: Phase 2 precision, recall and F1-score by tactic on the test split

6.1.3 Baseline model

The TF-IDF and logistic regression baseline reached a 5-fold cross-validation accuracy of 97.50% (± 0.45) and a scam-class F1 of 0.9810 (± 0.0034). On the external test set, its accuracy fell to 84.76%, with a scam-class F1 of 0.9002 but a legitimate-class F1 of only 0.6780 (Table 6.3). It found most scams (recall 0.92), but 36 of the 96 legitimate messages were flagged as scams, while 21 of the 278 scam messages were missed. The drop from cross-validation to the external set shows that performance on

the training distribution overstates performance on new messages. Because the baseline and DistilBERT were evaluated on different test sets, their scores are not compared directly.

Table 6.3: Baseline model results on the external test set

Class	Precision	Recall	F1-score	Support
Legitimate	0.7407	0.6250	0.6780	96
Scam	0.8771	0.9245	0.9002	278
Accuracy			0.8476	374
Macro F1			0.7891	

The threshold was 0.48. Of the 96 legitimate messages, 60 were classified correctly and 36 as scams; of the 278 scam messages, 257 were classified correctly and 21 as legitimate.

6.1.4 Error analysis

Reading the misclassified test messages showed two patterns. Messages with explicit fraud signals, such as requests for an upfront fee or a processing charge, were classified correctly. Messages that mix legitimate-sounding wording with subtle red flags, such as "earn daily payouts with no prior skills", were sometimes classified as legitimate. This shows that the model relies partly on surface wording, which more carefully written scams could avoid. On the legitimate side, genuine HR messages that mention other candidates can trigger FOMO, which is why the combination rule suppresses a lone FOMO detection when Phase 1 is confident that a message is legitimate (Section 4.4.3).

6.1.5 Outcome of the hypotheses

Table 6.4 reports the outcome of the hypotheses of Section 1.6.2.

Table 6.4: Outcome of the hypotheses

Hypothesis	Measure	Result	Outcome
H1 Scam-class F1 of at least 0.85	Scam-class F1 (Phase 1)	0.9065	Supported

H2 Macro F1 of at least 0.80	Macro F1 (Phase 2)	0.8345	Supported
H3 Urgency pressure and social proof detected better than FOMO and sunk-cost influence	Per-tactic F1 (Phase 2)	0.93 and 0.91 against 0.86 and 0.64	Supported

Both models met the hypothesis targets. Phase 1 also reached the target F1 of 0.90 set in the project proposal (0.9065), but Phase 2 did not (macro F1 0.8345). Because of the weaknesses of the data splits described in Section 3.2.6, the results may be somewhat optimistic; this is discussed in Chapter 7.

6.2 Product and Technical Results

The BSTD service was built as designed in Chapter 4. Table 6.5 summarizes the technical results.

Table 6.5: Technical results of the service

Item	Result
Endpoints	Eight endpoints implemented (Table 4.6), including /classify, /classify-image, /ocr-only and the history endpoints, reachable through the API gateway
Input modes	Built-in chat, pasted text and screenshots (EasyOCR with Tesseract fallback)
Output	Three-tier verdict, detected tactics, confidence, word importance and plain-language warning
Model loading	Both models loaded once at start-up; EasyOCR loaded on the first image request
Automated tests	20 tests: 18 passed and 2 integration tests skipped (Table 5.4)
Scenario tests (Table 5.5)	All four scenarios gave the expected result
OCR example	Screenshots converted to text and classified (Figures 4.3 and 6.5)

6.3 User or Stakeholder Validation

No study with job seekers or other users of the application was carried out for this component. The research questions concern how well scam messages and manipulation tactics can be detected, which is answered by the held-out experiment of Section 6.1; a user study would have needed ethical clearance and a complete release of the whole solution, which was not available during the project period. Whether job seekers understand and act on the warnings therefore remains untested and is listed as a limitation in Chapter 7 and as future work in Chapter 8.

The need for the component was, however, confirmed before development. A preliminary survey carried out from 2 to 9 March 2026 collected 78 responses from job seekers, undergraduate students and working professionals in Sri Lanka. Of the respondents, 85% had seen suspicious job advertisements and 68% had received suspicious messages from someone claiming to be a recruiter, most often on WhatsApp, LinkedIn and Telegram. The most common suspicious behaviour reported was a request for money or a processing fee, and 82% believed that recruitment scams are increasing. When asked about an automated tool, 58% said that they would use it or might use it, and the most requested feature was an explanation of why a message is suspicious. These results support the focus of the component on recruiter messages and on plain-language explanations. They validate the problem, not the finished product.

The solution was also reviewed with the external validator, Mr. Saman Gunawardana, who is working with the team. He considered the work good as a research project but saw its commercial position as uncertain. His comments concern the complete solution rather than this component alone; they and the response of the team are listed in Table 6.6.

Table 6.6: Feedback from the external validator and the response of the team

Feedback	Response of the team
How will job seekers be onboarded when many job applications already exist in the market?	Being addressed as part of the commercialisation plan of the group
What is the unique selling point, and why would a job seeker use or invest in the system?	The team is defining a clearer value proposition. For this component, the distinctive feature is tactic-level scam warnings inside recruiter conversations, which existing job platforms do not offer
The four components should work as one integrated system, not as separate features added to it	The integration of the components into one system is being strengthened, for example by connecting the communication-scam indicator to the recommendation component (Section 5.3.3)
The work is strong as research, but its commercial position is in a grey area	Acknowledged; commercialisation is addressed in the group report

6.4 Integration Results

In the built-in chat, the chat-management service calls /classify for each text message sent by a recruiter to a job seeker. Figure 6.3 shows a message that uses FOMO and urgency wording ("most people in ur batch already confirmed n paid … reply asap before spots close"). It is delivered to the job seeker with the red warning strip, the tactic labels FOMO and Urgency and the report button, and a "Possible scam message" entry is added to the job seeker's notification list through RabbitMQ. The recruiter's name is hidden in the figure.

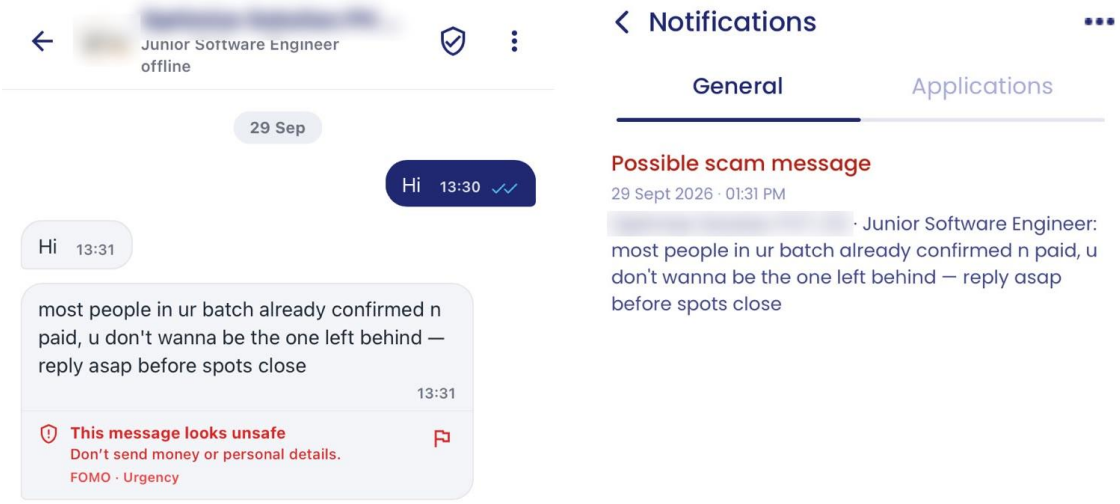


Figure 6.3: Warning shown to the job seeker in the built-in chat and the matching notification

In the Detect section, pasted text and screenshots return the verdict, the signal strength, the detected tactics, a warning and the most influential words. Figure 6.4 shows a genuine follow-up message after an interview, classed as LEGITIMATE with no tactic detected, and an upfront-fee request, classed as SCAM with urgency pressure. The highlighted words show which words most influenced the decision; for a legitimate message they are not scam words.

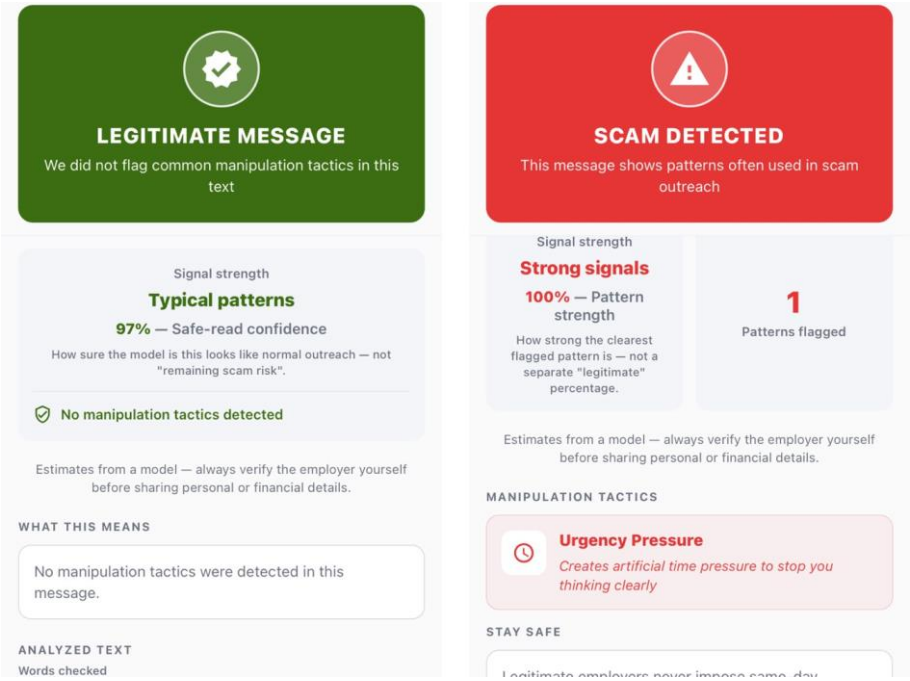


Figure 6.4: Results of a legitimate and a scam message in the Detect section

Figure 6.5 shows a screenshot of a scam message offering remote work with a guaranteed monthly income. The text was extracted by OCR and the message was classed as SCAM by the Phase 1 score, with no tactic above its threshold. Some of the flagged words, such as "jun" and "e8", come from the header of the screenshot rather than the message, which shows that OCR also reads text around the message.

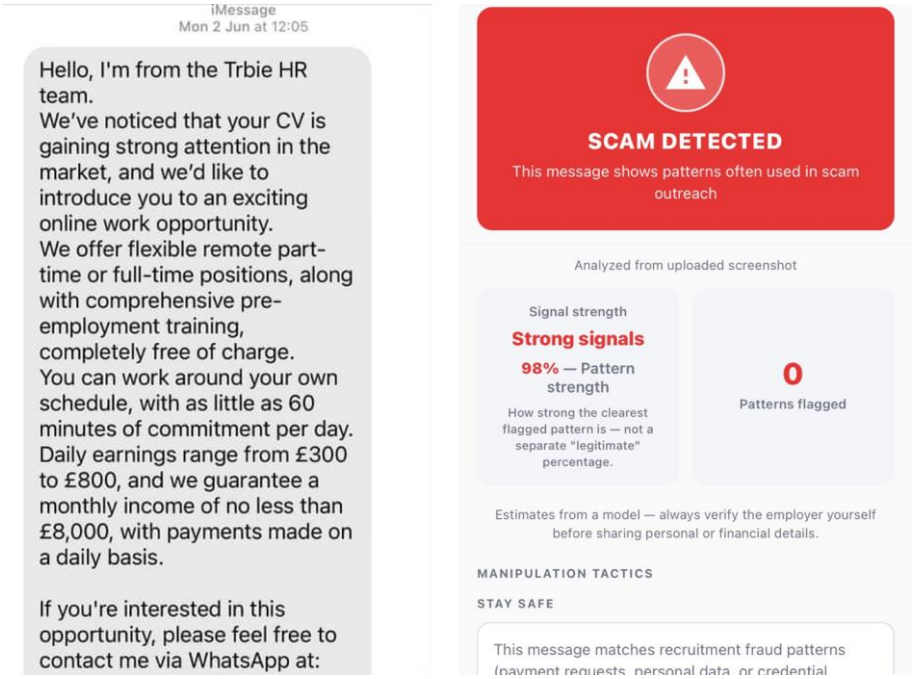


Figure 6.5: Checking a screenshot: the uploaded image and the result

Integration with the recommendation component is not complete. The recommender accepts the communication-scam indicator, but no part of the system fills it yet, so every job is treated as "Not Scam" (Section 5.3.3). A complete run from a flagged conversation to a changed job ranking was therefore not possible, and this is listed as a limitation in Chapter 7.

6.5 Summary of Findings

Table 6.7 summarizes the findings against the research questions

Table 6.7: Summary of findings

Question	Finding
RQ1 How can the datasets be built without leakage?	A binary dataset of 7,433 messages and a tactic dataset of 3,840 messages were built from public, synthetic and real sources, with a group-aware split for Phase 2. The Phase 1 split was not grouped and the phases were not separated, so leakage was reduced but not removed.
RQ2 How well does DistilBERT separate scam from legitimate messages?	Accuracy of 93.82% and scam-class F1 of 0.9065 on 744 test messages; precision is high for both classes, and scam recall is 0.88.

RQ3 How accurately are the tactics detected, and which are hardest?	Macro F1 of 0.8345 and micro F1 of 0.8722. Urgency pressure and social proof exceed 0.90; FOMO is hardest (F1 0.64, recall 0.54).
RQ4 How can all input modes be handled and explained?	One service handles chat, pasted text and screenshots and returns a three-tier verdict with a plain-language warning; the chat warns the job seeker on SUSPICIOUS and SCAM messages. User understanding of the warnings was not tested.

7 DISCUSSION

This chapter interprets the results of Chapter 6, compares them with related work and existing products, states the research and product contributions, examines the limitations and threats to validity, and considers the implications of the work.

7.1 Interpretation of Findings

The Phase 1 model separates scam from legitimate recruiter messages with an accuracy of 93.82%. Its precision is high for both classes, so legitimate messages are rarely flagged, but its scam recall of 0.88 means that roughly one scam message in eight is missed. For a protective tool, a missed scam is the more costly error, which is one reason for the three-tier verdict: a message with a moderate scam probability is shown as SUSPICIOUS rather than passed as legitimate, and any detected tactic makes a message SCAM even when its Phase 1 probability is low. The small difference between the validation F1 (0.9175) and the test F1 (0.9065) suggests a stable model, although the ungrouped Phase 1 split means that the test score may still be optimistic (Section 7.5.1).

The tactic results show a clear pattern. Urgency pressure and social proof, which are expressed through explicit and repeated wording such as deadlines, "today only", or references to other applicants and endorsements, are detected with F1-scores above 0.90. Sunk-cost influence, although it has the fewest examples, is also detected well (F1 0.86), probably because it is tied to recognisable phrases about money or effort already spent. FOMO is the exception: precision is acceptable (0.77), but recall is low (0.54). FOMO is expressed in more varied language than the other tactics, often overlaps with urgency in the same message, and in genuine HR messages can look similar to ordinary statements about other candidates. The finding of Anagha et al. [1], who could not reliably identify FOMO cues in survey data, suggests that this difficulty lies partly in the concept itself and not only in the model.

The near-identical validation and test macro F1 (0.8346 and 0.8345) indicates that the per-tactic thresholds were not over-fitted to the validation split. The error analysis, however, shows that the model relies partly on surface wording: messages that mix legitimate phrasing with subtle red flags can pass as legitimate. This is the main weakness that a scammer aware of the system could exploit.

The baseline shows that simple word features catch most scams but struggle with legitimate recruiter messages: more than a third of the legitimate messages in its external test set were flagged. This supports the use of a contextual model and the collection of more legitimate recruiter messages.

7.2 Comparison with Literature Baselines and Existing Solutions

A direct numerical comparison with earlier work is not possible, because no published system performs tactic-level detection in recruitment-scam conversations and the studies that are closest use different data and tasks. Table 7.1 therefore compares the approaches and reports a numerical result only where it was taken from the original publication.

Table 0.1: Comparison with the closest studies and products

Work	Task and input	Reported result	Tactics	User explanation
Vidros et al. [5]	Fake job advertisements (EMSCAD)	Advertisement classifiers	No	No
Taneja et al. [16]	Fake job advertisements	Transformer (Fraud-BERT) detector	No	No
Ghourabi [3]	SMiShing SMS	BERT-based detector	No	No
Shinde et al. [4]	Scam SMS images through OCR	94.13% accuracy (KNN-Flatten)	No	No
Tan et al. [17]	Instant-messaging scams	LLM-based warnings on 902 conversations	No	Yes
Anagha et al. [1]	Survey of job-scam victims	Urgency linked to payment; FOMO not reliably identified	Theory only	Not applicable

Google Messages [18]	SMS, MMS and RCS	Likely-scam warning	No	Generic
This research (BSTD)	Recruiter messages: chat, pasted text, screenshots	93.82% accuracy; tactic macro F1 0.8345	Yes (four)	Plain-language, names the tactic

Compared with advertisement-level detectors [5], [16], the component works at a different stage of the scam, after the first contact, and is therefore complementary rather than competing. Compared with message-level detectors [3], [4], it adds tactic labels and explanations for job seekers; the accuracy of Shinde et al. [4] is higher than the Phase 1 accuracy reported here, but it was measured on a different, non-recruitment SMS dataset and for a binary task, so the figures cannot be compared directly. Compared with the ASR framework of Tan et al. [17], which also gives real-time warnings, the component uses a much smaller model that can run on a modest server and names the specific tactic. Compared with commercial products such as Google Messages Scam Detection [18] and Truecaller [19], it is designed for recruitment conversations, works on messages from any platform through pasting or screenshots, and explains the manipulation instead of only warning. The main finding of the behavioural literature, that FOMO is hard to pin down [1], is reproduced here in an automated setting.

7.3 Research Contribution

The research makes three contributions. First, it turns the four-tactic taxonomy of recruitment-scam manipulation [1] into a working automated classifier and reports, for each tactic, how well it can be detected; to the best of the author's knowledge, this is the first such classifier for recruitment-scam messages. Second, it shows that intermediate-task transfer from binary scam detection to tactic detection [9], together with per-tactic thresholds, gives a tactic classifier with a macro F1 of 0.8345 from a modest amount of labelled tactic data. Third, it provides evidence that tactics with

explicit wording (urgency pressure, social proof) are much easier to detect than FOMO, which informs both future datasets and the design of warnings.

7.4 Product Contribution

The product contribution is a working service that brings scam detection into the conversation stage of recruitment. It accepts messages from the built-in chat, from any external platform by pasting, and from screenshots through OCR; it returns a three-tier verdict, the detected tactics and a plain-language warning; and in the built-in chat it warns the job seeker without blocking the conversation. Within the group solution, it covers the stage that the advertisement and employer checks cannot see, and it provides the interface through which a communication-scam indicator can be passed to the fraud-aware recommender.

7.5 Limitations and Threats to Validity

The limitations are grouped by the type of threat that they pose to the conclusions, together with their likely effect and how they can be addressed.

7.5.1 Internal validity

The Phase 1 split was stratified but neither deduplicated nor grouped, Phase 2 validation and test messages were not excluded from Phase 1 training, and the Phase 2 test set contains back-translated copies of some messages (Section 3.2.6). Each of these can make the reported results optimistic. A retraining with deduplication, a group-aware split for both phases and cross-phase exclusion is planned, and its results should be treated as the reference figures once available. Until such an evaluation is carried out, the extent of this effect cannot be measured.

7.5.2 Construct validity

The tactic labels were assigned by one person and verified with an industry data scientist and a large language model, but agreement between annotators was not measured. The four tactics also come from a single taxonomy [1]; other manipulation techniques, such as authority impersonation without social proof, are not separately modelled. Measuring agreement with Cohen's kappa on a sample of messages would strengthen the labels.

7.5.3 External validity

Most of the data are public corpora and synthetic messages; only 718 real recruitment messages were collected, so performance on real conversations may differ. The model covers English only, while many Sri Lankan job seekers receive Sinhala, Tamil or code-mixed messages. Messages are classified one at a time and truncated to 128 tokens, so tactics spread across a long message or a whole conversation may be missed.

7.5.4 Conclusion validity

Results come from a single test split, and confidence intervals were not computed. Small differences, for example between sunk-cost influence and social proof, should therefore not be over-interpreted, although the gap between FOMO and the other tactics is large enough to be meaningful.

7.5.5 Product limitations

No user study was carried out, so whether job seekers understand and act on the warnings is unknown. The communication-scam indicator is not yet connected to the recommendation component. Images sent in the chat are not checked, and the security settings of the service (any-origin CORS and acceptance of invalid TLS certificates) need tightening before public deployment. The library versions are not pinned, which affects exact reproducibility.

7.6 Implications

For job seekers, the component offers a way to check a recruiter message from any platform and to see, in plain language, why it may be a scam. For employment platforms in Sri Lanka, such as LinkedIn, ikman.lk and TopJobs.lk, the results show that conversation-level checks can complement advertisement moderation. For authorities such as the Computer Crimes Investigation Division, tactic labels on reported messages could help to identify recurring scam scripts. For researchers, the per-tactic results suggest that FOMO needs better definitions and more varied training examples, and that leakage-aware splits are essential when synthetic and augmented data are used.

The work is continuing with the external validator, Mr. Saman Gunawardana, who is reviewing the component and has suggested changes that are being incorporated.

7 CONCLUSION AND FUTURE DIRECTIONS

This chapter concludes the dissertation, reviews the achievement of the objectives, summarizes the individual contributions and sets out directions for future work.

8.1 Conclusion

Online recruitment scams are usually completed in private conversations after the first contact, where advertisement-level detectors cannot help and where scammers use psychological pressure to obtain money and personal data. This research designed, built and evaluated the Behaviour-Inspired Semantic Tactic Detection (BSTD) component, which checks recruiter messages and names the manipulation tactics that they use.

A DistilBERT model was trained in two phases. Phase 1 classifies a message as scam or legitimate and reached an accuracy of 93.82% and a scam-class F1 of 0.9065 on 744 held-out messages. Phase 2, initialised from the Phase 1 encoder, detects urgency pressure, social proof, FOMO and sunk-cost influence, and reached a macro F1 of 0.8345 and a micro F1 of 0.8722 on 758 held-out messages. Urgency pressure and social proof were detected most reliably, while FOMO remained the hardest tactic, in line with the behavioural literature. All three working hypotheses were supported, and Phase 1 also reached the higher target of the project proposal, although Phase 2 did not, and the results may be somewhat optimistic because of weaknesses in the data splits.

As a product, the component accepts messages from the built-in chat, from any external platform by pasting and from screenshots through OCR, combines the two models into a three-tier verdict, and explains each decision in plain language. In the built-in chat it warns the job seeker without blocking the conversation. Together with the advertisement and employer checks of the group solution, it extends protection to the stage of a recruitment scam in which money is actually lost.

8.2 Achievement of Objectives

Table 8.1 summarizes the achievement of the objectives of Section 1.5.2.

Table 7.1: Achievement of the objectives of Section 1.5.2

Objective	Status	Evidence
O1 Review related work and identify the gaps	Achieved	Chapter 2; gaps G1 to G6 in Table 2.4
O2 Build the binary and tactic datasets with leakage-aware splits	Achieved with limitations	Section 3.2; group-aware split for Phase 2, but the Phase 1 split was not grouped and the phases were not separated
O3 Fine-tune DistilBERT for binary classification	Achieved	Section 6.1.1; accuracy 93.82%, scam-class F1 0.9065
O4 Tactic classifier with per-tactic thresholds	Achieved	Section 6.1.2; macro F1 0.8345, micro F1 0.8722
O5 Three input modes and plain-language warnings	Achieved	Chapters 4 and 5; Sections 6.2 and 6.4; user understanding not tested
O6 Real-time service and integration	Partly achieved	Chat integration working (Section 6.4); the indicator for the recommendation component is provided but not yet connected

8.3 Individual Research and Product Contributions

The author's individual contributions to the research are the collection of 718 real recruitment messages and the preparation of the binary and tactic datasets, including back-translation of real messages and the tactic labels; the design and training of the two-phase DistilBERT model with intermediate-task transfer and per-tactic thresholds; and the evaluation of both phases on held-out data, with an analysis of which tactics are hardest to detect and why.

The individual contributions to the product are the FastAPI scam-detection service with its eight endpoints; the OCR pipeline with EasyOCR and the Tesseract fallback; the three-tier combination rule and its unit tests; the plain-language warnings based on word-level explanations; and the integration with the chat-management service so that job seekers are warned in real time.

8.4 Future Directions

Several directions follow from the results and limitations of this work.

- Retrain both phases on deduplicated data with group-aware splits and cross-phase exclusion, keep back-translated copies out of the validation and test sets, and report the resulting figures as the reference results.
- Collect more real recruitment conversations and extend the model to Sinhala, Tamil and code-mixed messages, which many Sri Lankan job seekers receive.
- Analyse whole conversations instead of single messages, so that escalation from urgency to sunk-cost pressure can be detected, and handle messages longer than 128 tokens.
- Improve the detection of FOMO with clearer definitions, more varied examples and measured annotator agreement.
- Carry out a user study with job seekers to test whether the warnings are understood and change behaviour.
- Connect the communication-scam indicator to the fraud-aware recommendation component, check images sent in the chat, and tighten the security settings of the service.
- Continue the review with the external validator and incorporate the suggested changes.

REFERENCES

[1] G. Anagha, V. D. Agrawal, G. Sarkar, K. Vemuri, and S. K. Shukla, "Modeling behavioral signals in job scams: A human-centered security study," arXiv:2601.19342, 2026.

[2] A. Pitumpe and A. Rahmati, "Anansi: Scalable characterization of message-based job scams," arXiv:2602.24223, 2026.

[3] A. Ghourabi, "SM-Detector: A security model based on BERT to detect SMiShing messages in mobile environments," Concurrency and Computation: Practice and Experience, vol. 33, no. 24, 2021.

[4] A. Shinde, E. Q. Shahra, S. Basurra, F. Saeed, A. A. AlSewari, and W. A. Jabbar, "SMS scam detection application based on optical character recognition for image data using unsupervised and deep semi-supervised learning," Sensors, vol. 24, no. 18, Art. no. 6084, 2024, doi: 10.3390/s24186084.

[5] S. Vidros, C. Kolias, G. Kambourakis, and L. Akoglu, "Automatic detection of online recruitment frauds: Characteristics, methods, and a public dataset," Future Internet, vol. 9, no. 1, Art. no. 6, 2017, doi: 10.3390/fi9010006.

[6] R. B. Cialdini, Influence: The Psychology of Persuasion, rev. ed. New York, NY, USA: Harper Business, 2006.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[9] Y. Pruksachatkun, J. Phang, H. Liu, P. M. Htut, X. Zhang, R. Y. Pang, C. Vania, K. Kann, and S. R. Bowman, "Intermediate-task transfer learning with pretrained language models: When and why does it work?," in Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 5231-5247, doi: 10.18653/v1/2020.acl-main.467.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[11] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135-1144, doi: 10.1145/2939672.2939778.

[12] M. A. Uddin and I. H. Sarker, "An explainable transformer-based model for phishing email detection: A large language model approach," arXiv:2402.13871, 2024.

[13] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in Proc. 11th ACM Symp. Document Engineering (DocEng), 2011.

[14] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in Proc. 54th Annu. Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 86-96, doi: 10.18653/v1/P16-1009.

[15] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, Art. no. 15, 2012, doi: 10.1145/2382577.2382579.

[16] K. Taneja, J. Vashishtha, and S. Ratnoo, "Fraud-BERT: Transformer based context aware online recruitment fraud detection," Discover Computing, vol. 28, no. 1, 2025, doi: 10.1007/s10791-025-09502-8.

[17] X. W. Tan, K. See, and S. Kok, "Anticipate, simulate, reason (ASR): A comprehensive generative AI framework for combating messaging scams," arXiv:2507.17543, 2025.

[18] Google, "New AI-powered scam detection features to help protect you on Android," The Keyword (Google Blog), Mar. 4, 2025. [Online]. Available: https://blog.google/security/new-ai-powered-scam-detection-features/

[19] The Paypers, "Truecaller launches fraud protection feature," Apr. 2023. [Online]. Available: https://thepaypers.com/fraud-and-fincrime/news/truecaller-launches-fraud-protection-feature

[20] Personal Data Protection Act, No. 9 of 2022, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022. [Online]. Available: https://documents.gov.lk/view/acts/2022/3/09-2022_E.pdf

GLOSSARY

Table G.1: Glossary of terms

Term	Meaning
Back-translation	Translating a sentence into another language and back again to create a paraphrased copy
Combination rule	The rule that turns the outputs of the two models into a SCAM, SUSPICIOUS or LEGITIMATE verdict
Data leakage	When information from the test data, or near-copies of test messages, is available during training, which makes results look better than they are
DistilBERT	A smaller and faster version of the BERT language model, produced by knowledge distillation
Exact-match accuracy	The share of messages for which the full set of tactics is predicted correctly
F1-score	The harmonic mean of precision and recall
FOMO	Fear of missing out: pressure created by suggesting that an opportunity is limited or will be lost to others
Group-aware split	A data split that keeps related messages, such as a message and its back-translations, in the same subset
Hamming loss	The share of individual tactic decisions that are wrong
Intermediate-task transfer	Fine-tuning a model on a related task before fine-tuning it on the target task
Macro and micro F1	Macro F1 averages the F1-scores of the labels equally; micro F1 pools all decisions before computing F1
Multi-label classification	Classification in which a message can receive several labels at once
Optical character recognition	Converting the text in an image into machine-readable text
Social proof	In this dissertation, the use of fake authority or endorsements to appear legitimate
Sunk-cost influence	Pressure to pay more because money or effort has already been spent
Threshold	The probability above which a tactic is flagged
Urgency pressure	Pressure created by artificial deadlines or time limits

APPENDICES

Appendix A Extended Results and Technical Specification

This appendix gathers material that supports Chapters 3 to 6 and is too detailed for the main text.

A.1 Settings used

Table A.1: Settings of the data, models and service

Item	Value
Phase 1 data	7,433 messages from EMSCAD, the UCI SMS Spam Collection, 1,000 synthetic messages and 718 real messages; stratified 80/10/10 split, seed 42; test set 744 (491 legitimate, 253 scam)
Phase 2 data	3,840 messages; group-aware 70/10/20 split (GroupShuffleSplit), seed 42; test set 758
Back-translation	105 real messages through Sinhala, Tamil and French; 304 copies generated, 217 kept
Model	distilbert-base-uncased; Phase 2 initialised from the Phase 1 encoder
Training	AdamW, learning rate 2e-5, weight decay 0.01, linear schedule with 10% warm-up, gradient clipping 1.0, dropout 0.3; batch size 32 (Phase 1) and 16 (Phase 2); 3 and 6 epochs; best epochs 3 and 5
Tactic thresholds	Urgency pressure 0.25, FOMO 0.30, sunk-cost influence 0.25, social proof 0.45; searched from 0.25 to 0.60 in steps of 0.05 on the validation split; a tactic is flagged when its probability exceeds the threshold
Combination rule	Lone FOMO with $p < 0.40$ gives LEGITIMATE; any tactic gives SCAM; $p \geq 0.75$ gives SCAM; $0.40 \leq p < 0.75$ gives SUSPICIOUS; otherwise LEGITIMATE
OCR	EasyOCR, dropping tokens with confidence below 0.12; Tesseract fallback when the cleaned text is shorter than 18 characters or EasyOCR fails, keeping the longer result
Limits	128 tokens per message; first 2,000 characters from chat; 15-second chat time-out

A.2 Preliminary survey

The preliminary survey summarized in Section 6.3 was carried out from 2 to 9 March 2026 and received 78 responses. It asked about age group and current status, online job searching and the platforms used, exposure to suspicious job advertisements and recruiter messages, the platforms on which suspicious messages were received, the

suspicious behaviours noticed, requests for payment, confidence in identifying fake offers, whether scams are increasing, willingness to use an automated detection tool, the features that would help most, and the biggest risk of applying for jobs online. Table A.2 lists the key findings.

Table A.2: Key findings of the preliminary survey (n = 78)

Finding	Result
Have seen suspicious job advertisements	85%
Have received suspicious recruiter messages	68%
Most common suspicious behaviour	Asking for money or a processing fee
Most common platforms for suspicious messages	WhatsApp, LinkedIn, Telegram
Believe recruitment scams are increasing	82%
Would use an AI scam detection tool	58% yes or maybe
Most requested feature	An explanation of why a message is suspicious

Appendix B Academic Integrity and AI Use Disclosure

Generative AI was used in this work as follows, in accordance with the module policy:

- Claude (Anthropic) was used as a second check when verifying the tactic labels, alongside an industry data scientist. The final labels were decided by the author.
- Claude was used for guidance on evaluation design, for checking analysis scripts, for language editing of this dissertation, and for help in preparing the diagrams.

No AI tool generated the experimental results. All models were trained, all results were produced by the author's own code, and all figures, numbers and citations in this dissertation were checked by the author, who remains responsible for their accuracy.